

UNIT-II

Adversarial Search: Games

Adversarial Search is a search technique used when **two or more agents compete against each other.**

In normal search problems like BFS, DFS, or A*, the agent searches for a path to reach a goal.

But in games, there is an opponent who is also thinking and trying to stop us from winning.

So, adversarial search means:

Searching for the best move when another player is trying to defeat us.

1. Meaning of “Adversarial”

Adversary means **opponent/enemy/competitor.**

In AI games:

Player	Goal
---------------	-------------

AI agent	Wants to win
----------	--------------

Opponent	Wants AI to lose
----------	------------------

Example:

In chess, if AI moves a pawn, the opponent will not stay silent. The opponent will make a move to create difficulty for AI.

So, AI must think:

“If I make this move, what will the opponent do next?”

That is the main idea of adversarial search.

2. Why Games are Important in AI?

Games are useful in AI because they require:

- Thinking ahead
- Planning
- Decision-making

- Handling competition
- Choosing the best move
- Predicting opponent actions

Examples:

Game	AI Decision
-------------	--------------------

Chess	Which piece should I move?
-------	----------------------------

Tic-Tac-Toe	Where should I place X?
-------------	-------------------------

Checkers	Which move gives advantage?
----------	-----------------------------

Ludo	Which token should I move?
------	----------------------------

Carrom	Which coin should I hit?
--------	--------------------------

Games are good examples of intelligent behavior because the AI must make smart decisions.

3. Difference Between Normal Search and Game Search

Normal Search

Example: Finding a path from home to college.

There is no enemy.

The agent only searches for the best path.

Example algorithms:

- BFS
- DFS
- Uniform Cost Search
- A*

Adversarial Search

Example: Playing chess.

There is an opponent.

The agent must think about both its own move and the opponent's move.

Example algorithms:

- Minimax algorithm
- Alpha-Beta pruning

4. Simple Real-Life Explanation

Imagine two students are playing chess.

Student A thinks:

“If I move my queen here, I may attack.”

But then Student A also thinks:

“What will Student B do after that?”

Student B may block the attack or capture the queen.

So Student A must think multiple steps ahead.

This thinking process is called **adversarial search**.

5. Game Tree

In adversarial search, possible moves are represented as a **game tree**.

A game tree shows:

- Current game position
- Possible moves of AI
- Possible moves of opponent
- Future outcomes

Example in Tic-Tac-Toe:

Current State

```

  |
AI possible moves
 /  |  \
Move1 Move2 Move3
 |    |    |
Opponent replies
```

Each level of the tree represents one player's turn.

6. Important Terms in Game Search

1. Initial State

The starting position of the game.

Example:

In chess, the initial state is the board arrangement before the first move.

2. Player

The person or AI who makes a move.

Usually there are two players:

- MAX player
- MIN player

3. Actions

Legal moves available to a player.

Example:

In Tic-Tac-Toe, placing X in an empty cell is an action.

4. Result / Transition Model

Shows what happens after a move.

Example:

If AI places X in the center cell, the board changes.

5. Terminal State

The game-ending state.

Example:

- Win
- Lose
- Draw

6. Utility Value

A numerical value given to the final result.

Example:

Result Utility

AI wins +1

Draw 0

AI loses -1

In some games:

Result Utility

Win +10

Draw 0

Loss -10

Optimal Decisions in Games

Optimal decision means choosing the **best possible move** in a game.

In AI games, the agent should not select a move randomly. It should select a move after thinking:

“Which move gives me the best chance to win, assuming my opponent also plays intelligently?”

This is called **optimal decision-making in games**.

Meaning of Optimal Decision

An **optimal decision** is the best decision among all possible decisions.

In games, the AI must choose a move that gives the best outcome.

Example:

In Tic-Tac-Toe, if AI can win immediately by placing X in one box, that move is the optimal move.

```
X | X |  
O | O |  
  |  |
```

Here, if AI is X, the best move is:

```
X | X | X  
O | O |  
  |  |
```

AI wins immediately. So this is the **optimal decision**.

2. Why Optimal Decisions are Difficult in Games

In normal search problems, the AI only thinks about reaching the goal.

Example:

Find shortest path from home to college.

But in games, there is an opponent.

The opponent is also thinking:

“How can I stop the AI from winning?”

So AI must consider:

- Its own move
- Opponent's possible move
- Future moves
- Winning, losing, or drawing situations

That is why game decisions are more complex than normal search decisions.

3. Main Idea

In game playing, AI follows this idea:

Choose the move that gives the best result, even after the opponent makes the best possible response.

1. Mini-Max Algorithm

- Mini-max algorithm is a recursive or backtracking algorithm which is used in decision-making and game theory. It provides an optimal move for the player assuming that opponent is also playing optimally.
- Mini-Max algorithm uses recursion to search through the game-tree.
- Min-Max algorithm is mostly used for game playing in AI. Such as Chess, Checkers, tic-tac-toe, go, and various tow-players game. This Algorithm computes the minimax decision for the current state.
- In this algorithm two players play the game, one is called MAX and other is called MIN.
- Both the players fight it as the opponent player gets the minimum benefit while they get the maximum benefit.
- Both Players of the game are opponent of each other, where MAX will select the maximized value and MIN will select the minimized value.
- The minimax algorithm performs a depth-first search algorithm for the exploration of the complete game tree.
- The minimax algorithm proceeds all the way down to the terminal node of the tree, then backtrack the tree as the recursion.

Working of Min-Max Algorithm:

The working of the minimax algorithm can be easily described using an example. Below we have taken an example of game-tree which is representing the two-player game.

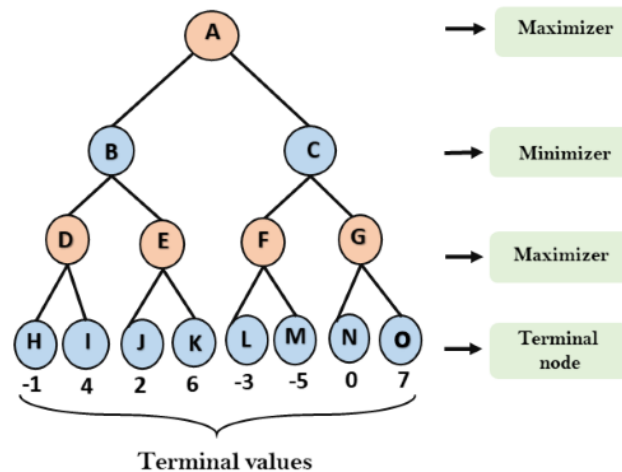
In this example, there are two players one is called Maximizer and other is called Minimizer.

Maximizer will try to get the Maximum possible score, and Minimizer will try to get the minimum possible score.

This algorithm applies DFS, so in this game-tree, we have to go all the way through the leaves to reach the terminal nodes.

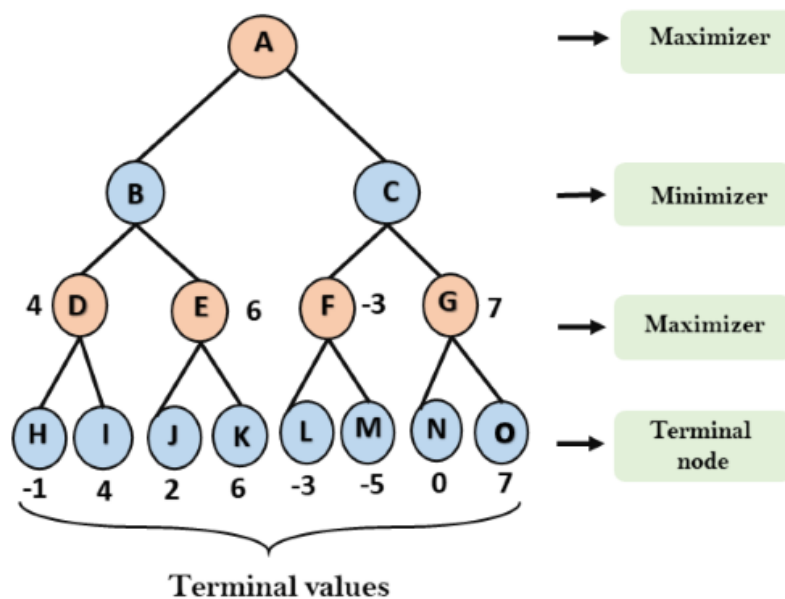
At the terminal node, the terminal values are given so we will compare those value and backtrack the tree until the initial state occurs. Following are the main steps involved in solving the two-player game tree:

Step-1: In the first step, the algorithm generates the entire game-tree and apply the utility function to get the utility values for the terminal states. In the below tree diagram, let's take A is the initial state of the tree. Suppose maximizer takes first turn which has worst-case initial value =- infinity, and minimizer will take next turn which has worst-case initial value = +infinity.



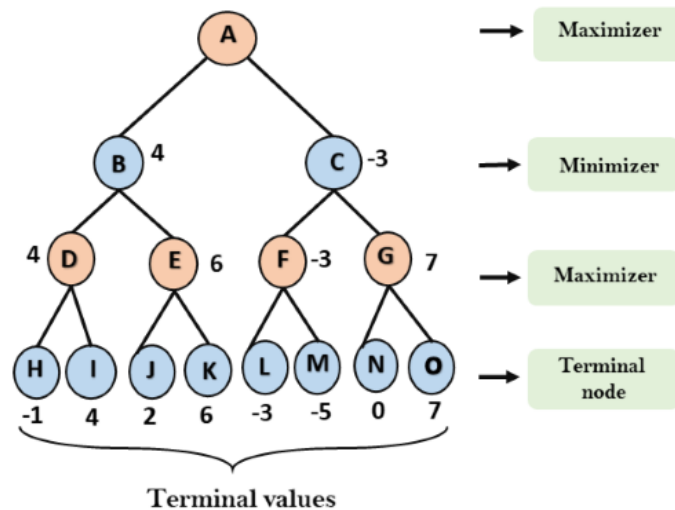
Step 2: Now, first we find the utilities value for the Maximizer, its initial value is $-\infty$, so we will compare each value in terminal state with initial value of Maximizer and determines the higher nodes values. It will find the maximum among the all.

- For node D $\max(-1, -\infty) \Rightarrow \max(-1, 4) = 4$
- For Node E $\max(2, -\infty) \Rightarrow \max(2, 6) = 6$
- For Node F $\max(-3, -\infty) \Rightarrow \max(-3, -5) = -3$
- For node G $\max(0, -\infty) = \max(0, 7) = 7$



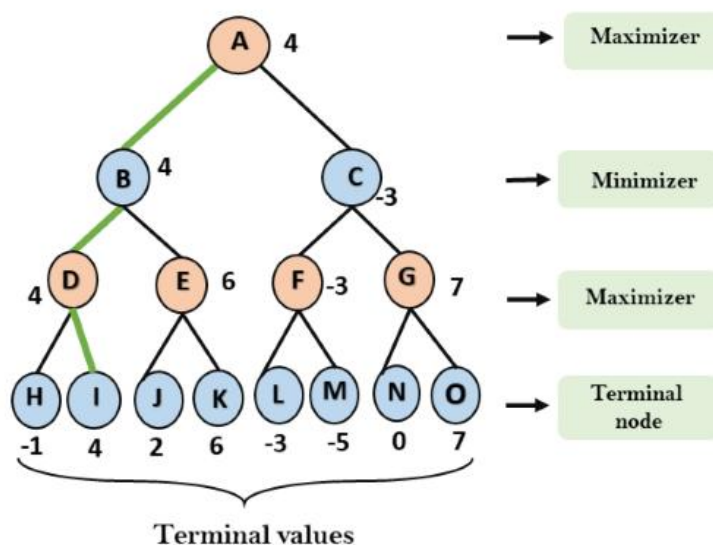
Step 3: In the next step, it's a turn for minimizer, so it will compare all nodes value with $+\infty$, and will find the 3rd layer node values.

- For node B = $\min(4, 6) = 4$
- For node C = $\min(-3, 7) = -3$



Step 4: Now it's a turn for Maximizer, and it will again choose the maximum of all nodes value and find the maximum value for the root node. In this game tree, there are only 4 layers, hence we reach immediately to the root node, but in real games, there will be more than 4 layers.

- For node A $\max(4, -3) = 4$



That was the complete workflow of the minimax two player game.

Properties of Mini-Max algorithm:

- **Complete-** Min-Max algorithm is Complete. It will definitely find a solution (if exist), in the finite search tree.
- **Optimal-** Min-Max algorithm is optimal if both opponents are playing optimally.
- **Time complexity-** As it performs DFS for the game-tree, so the time complexity of Min-Max algorithm is $O(b^m)$, where b is branching factor of the game-tree, and m is the maximum depth of the tree.
- **Space Complexity-** Space complexity of Mini-max algorithm is also similar to DFS which is $O(bm)$.

Limitation of the minimax Algorithm:

The main drawback of the minimax algorithm is that it gets really slow for complex games such as Chess, go, etc. This type of games has a huge branching factor, and the player has lots of choices to decide. This limitation of the minimax algorithm can be improved from **alpha-beta pruning** which we have discussed in the next topic.

2.Alpha-Beta Pruning

Alpha-beta pruning is a modified version of the minimax algorithm. It is an optimisation technique for the minimax algorithm.

As we have seen in the minimax search algorithm, the number of game states it must examine is exponential in the depth of the tree. Since we cannot eliminate the exponent, we can cut it in half. Hence, there is a technique by which, without checking each node of the game tree, we can compute the correct minimax decision, and this technique is called pruning. This involves two threshold parameters, Alpha and Beta, for future expansion, so it is called alpha-beta pruning. It is also called the Alpha-Beta Algorithm.

Alpha-beta pruning can be applied at any depth of a tree, and sometimes, it affects not only the tree leaves but also the entire sub-tree. The two-parameter can be defined as:

1. **Alpha:** The best (highest-value) choice we have found so far at any point along the path of Maximiser. The initial value of alpha is $-\infty$.
2. **Beta:** The best (lowest-value) choice we have found so far at any point along the path of Minimiser. The initial value of beta is $+\infty$.

The Alpha-beta pruning to a standard minimax algorithm returns the same move as the standard algorithm does, but it removes all the nodes that are not really affecting the final decision, making the algorithm slow. Hence, by pruning these nodes, it makes the algorithm faster.

Conditions for Alpha-beta Pruning

The main condition required for alpha-beta pruning is:

$$\alpha \geq \beta$$

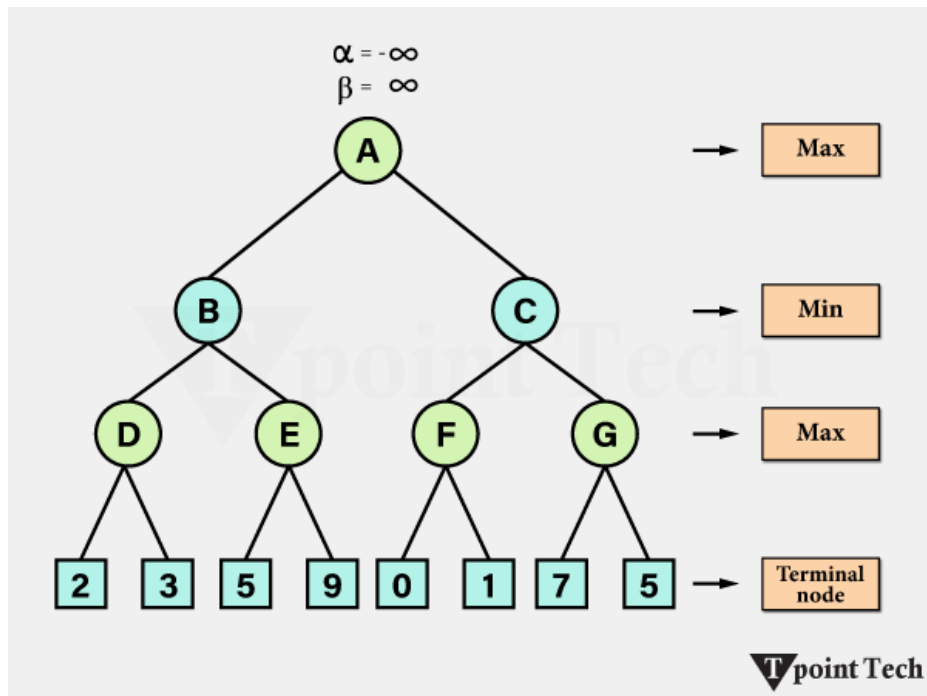
Key Points about Alpha-beta Pruning:

- The Max player will only update the value of alpha.
- The Min player will only update the value of beta.
- While backtracking the tree, the node values will be passed to the upper nodes instead of the values of alpha and beta.
- We will only pass the alpha and beta values to the child nodes.

Working of Alpha-Beta Pruning

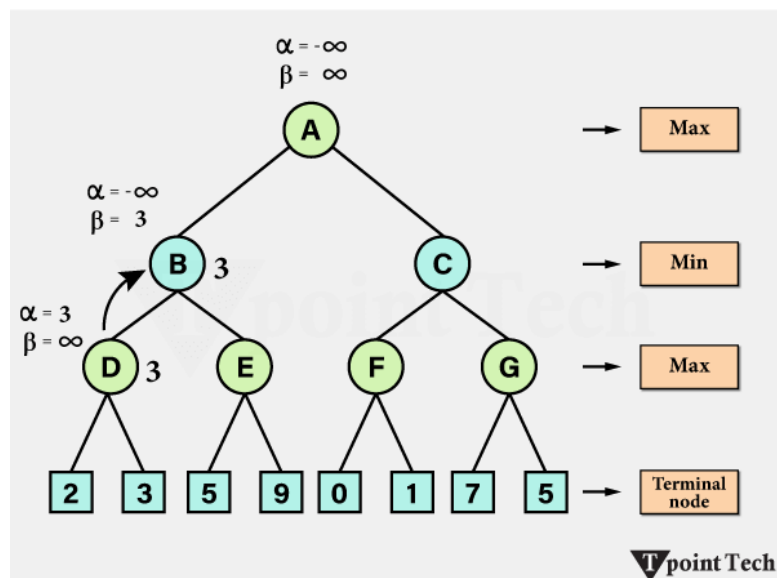
Let's take an example of a two-player search tree to understand the working of Alpha-beta pruning:

Step 1: At the first step, the Max player will start the first move from node A, where $\alpha = -\infty$ and $\beta = +\infty$; these values of alpha and beta are passed down to node B, where again $\alpha = -\infty$ and $\beta = +\infty$, and Node B passes the same value to its child D.



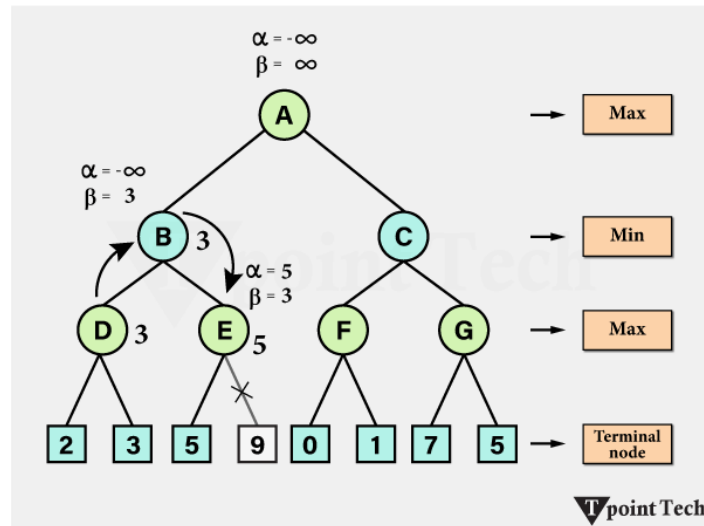
Step 2: At Node D, the value of α will be calculated as its turn for Max. The value of α is compared with firstly 2 and then 3, and the $\max(2, 3) = 3$ will be the value of α at node D, and the node value will also be 3.

Step 3: Now the algorithm backtracks to node B, where the value of β will change as this is a turn of Min. Now $\beta = +\infty$ will compare with the available subsequent nodes' value, i.e., $\min(\infty, 3) = 3$, hence at node B, now $\alpha = -\infty$, and $\beta = 3$.



In the next step, the algorithm traverses the next successor of Node B, which is Node E, and the values of $\alpha = -\infty$ and $\beta = 3$ will also be passed.

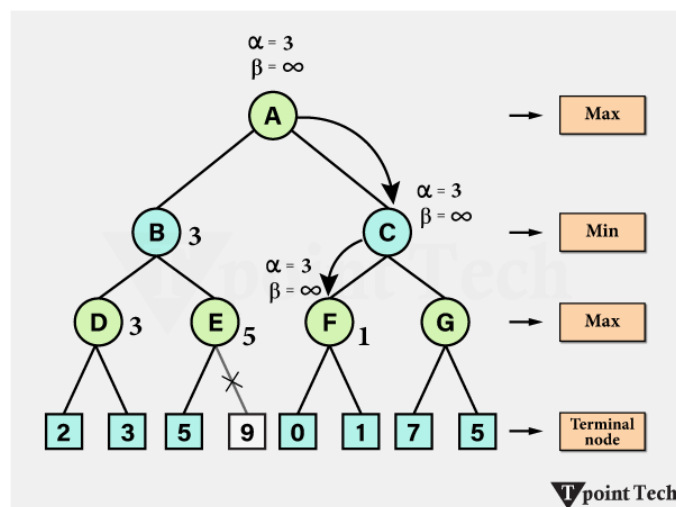
Step 4: At node E, Max will take its turn, and the value of α will change. The current value of α will be compared with 5, so $\max(-\infty, 5) = 5$, hence at node E $\alpha = 5$ and $\beta = 3$, where $\alpha \geq \beta$, so the right successor of E will be pruned, and the algorithm will not traverse it, and the value at node E will be 5.



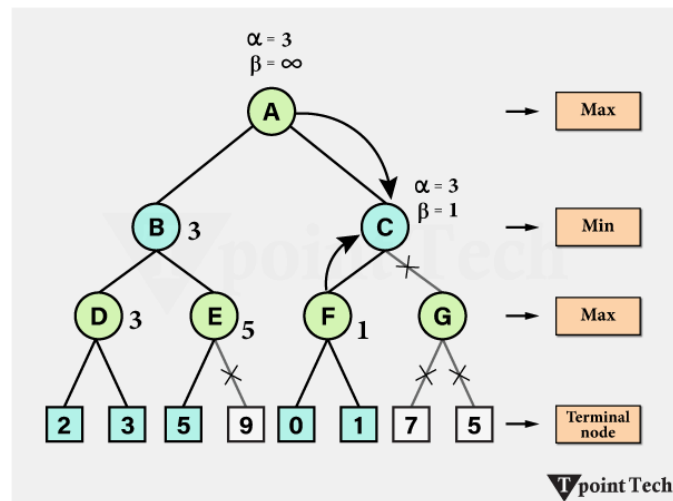
Step 5: At the next step, the algorithm again backtracks the tree from node B to node A. At node A, the value of α will be changed to the maximum available value of 3 as $\max(-\infty, 3) = 3$, and $\beta = +\infty$; these two values now pass to the right successor of A, which is Node C.

At node C, $\alpha = 3$ and $\beta = +\infty$, and the same values will be passed on to node F.

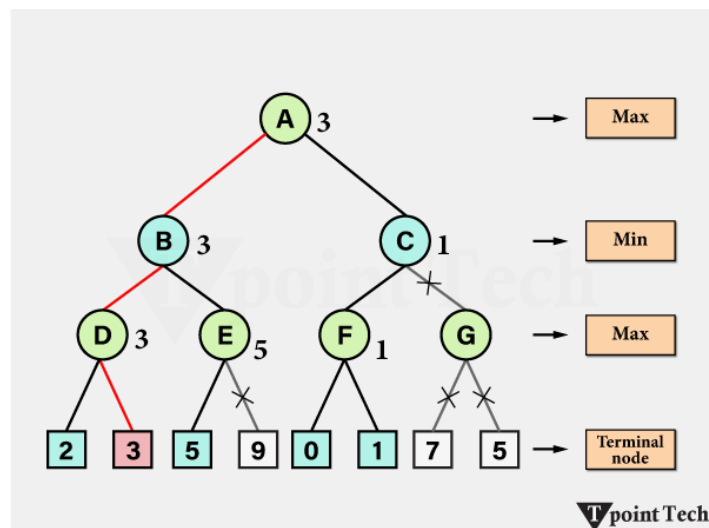
Step 6: At node F, again, the value of α will be compared with the left child, which is 0, and $\max(3, 0) = 3$, and then compared with the right child, which is 1, and $\max(3, 1) = 3$ still α remains 3, but the node value of F will become 1.



Step 7: Node F returns the node value 1 to node C, at C $\alpha = 3$ and $\beta = +\infty$; here, the value of β will be changed, and it will compare with 1, so $\min(\infty, 1) = 1$. Now at C, $\alpha = 3$ and $\beta = 1$, and again it satisfies the condition $\alpha > \beta$, so the next child of C, which is G, will be pruned, and the algorithm will not compute the entire sub-tree G.



Step 8: C now returns the value of 1 to A. Here, the best value for A is $\max(3, 1) = 3$. Following is the final game tree, which shows the nodes that are computed and the nodes that have never been calculated. Hence, the optimal value for the maximiser is 3 for this example.



Move Ordering in Alpha-Beta Pruning

The effectiveness of alpha-beta pruning is highly dependent on the order in which each node is examined. Move order is an important aspect of alpha-beta pruning.

It can be of two types:

1. **Worst ordering:** In some cases, the alpha-beta pruning algorithm does not prune any of the leaves of the tree and works exactly as the minimax algorithm. In this case, it also consumes more time because of alpha-beta factors, such as a move of pruning, which is called worst ordering. In this case, the best move occurs on the right side of the tree. The time complexity for such an order is $O(bm)$.
2. **Ideal ordering:** The ideal ordering for alpha-beta pruning occurs when lots of pruning happens in the tree, and the best moves occur at the left side of the tree. We apply DFS. Hence, it first searches the left of the tree and goes deep twice as the minimax algorithm does in the same amount of time. Complexity in ideal ordering is $O(bm/2)$.

Rules to Find a Good Ordering

The following are some rules to find good ordering in alpha-beta pruning:

- Occur the best move from the shallowest node.
- Order the nodes in the tree such that the best nodes are checked first.
- Use domain knowledge while finding the best move. Ex: for Chess, try order: captures first, then threats, then forward moves, backward moves.
- We can bookkeep the states, as there is a possibility that states may repeat.

Advantages and Disadvantages of Alpha-Beta Pruning

Advantages

- **Enhanced Efficiency Compared to Plain Minimax:** Alpha-Beta Pruning can considerably enhance the performance of the Minimax algorithm by drilling down the decision tree from an increasing number of nodes. The efficiency comes from pruning branches that do not affect the result. This means that the algorithm is only interested in the most promising paths, and since there can be many paths to the goal, this speeds up decision making in such cases as chess or checkers.
- **Applicability to Large Decision Trees:** This algorithm is especially useful for large search space games and decision problems. Removing unneeded branches permits deeper searching of the tree for the same computational budget. The depth advantage is very important for making better decisions, primarily in a competitive environment where precise foresight is essential.

Disadvantages

- **Dependency on the Order of Node Evaluation:** As an order controls the ordering of nodes in Alpha Beta Pruning, its efficiency depends greatly on that order. Therefore, if the algorithm processes the most promising moves first, so-called "node ordering", then the pruning effect is at its maximum. However, node ordering can also harm computational efficiency negatively if it results in suboptimal pruning. Consequently, proper implementation with heuristics or preprocessing is typically necessary to obtain optimal performance.
- **Computational Overhead for Deep and Complex Trees:** Alpha-Beta Pruning reduces the exploration of the nodes but cannot be efficient on highly complex trees with extremely high depth. In particular, the algorithm may still require an amount of computational effort that is at least as high as in the worst case if no good heuristics for ordering nodes are available. However, when the problem domain calls for such methods, Monte Carlo Tree Search may be a more appropriate way to solve such problems.

Imperfect Real-Time Decisions

Minimax can find the optimal move only when it searches the game tree deeply enough. In real games such as chess, the AI has limited time and cannot examine every possible future move.

Even with alpha-beta pruning, the game tree may still be too large.

Therefore, the AI must make a decision before completing the entire search.

An **imperfect real-time decision** is a decision made by an AI under limited time and computational resources, without completely searching the game tree.

The AI selects the best move found within the available time.

The word **imperfect** does not mean the move is necessarily wrong. It means the AI cannot guarantee that it is the theoretically optimal move.

Suppose a chess player has only **10 seconds** to make a move.

They cannot analyse every possible move until the end of the game. Instead, they:

1. Examine a few promising moves.
2. Think several moves ahead.
3. Estimate which position is better.
4. Select the best available move before time expires.

Game-playing AI works in the same way.

Why are real-time decisions necessary?

In complex games:

- Many legal moves are available.
- Each move creates many opponent responses.
- The game tree becomes extremely large.
- The decision must be made within a fixed time.
- Available memory and processing power are limited.

For example, assume each state has 20 possible moves.

Search depth Possible states

1	20
2	400
3	8,000
4	160,000
5	3,200,000

Thus, searching only a few additional levels greatly increases the computation.

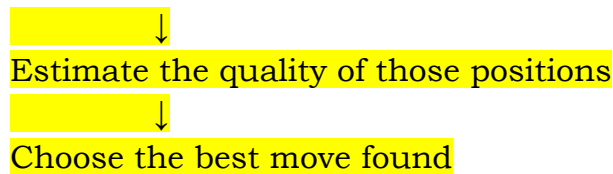
Main idea

Since the AI cannot reach terminal states such as win, lose or draw, it performs three important operations:

Search only to a limited depth



Stop at non-terminal positions



The two key concepts are:

1. **Cutoff test**
2. **Evaluation function**

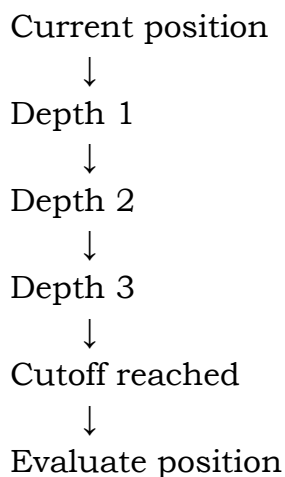
Cutoff test

A **cutoff test** determines when the search should stop before reaching the end of the game.

The search may stop when:

- A fixed depth is reached
- The time limit is nearly over
- A fixed number of nodes has been examined
- Available memory is insufficient

Example:



Evaluation function

An **evaluation function** estimates how good a non-terminal game position is.

It returns a numerical score.

Score	Meaning
--------------	----------------

Positive value	Position favours MAX
----------------	----------------------

Zero	Position is approximately equal
------	---------------------------------

Negative value	Position favours MIN
----------------	----------------------

Example:

+10 = very strong position for AI

+3 = slight advantage for AI

0 = equal position

-4 = advantage for opponent

-10 = very weak position for AI

Unlike the utility function, the evaluation function does not give the exact final outcome. It only estimates the likely quality of the current position.

Example:

Utility:

Win = +1

Draw = 0

Loss = -1

Evaluation:

Strong position = +8

Balanced position = 0

Weak position = -6

Depth-limited search

In a real-time game, the AI searches only up to a selected depth.

For example:

Search until depth 4

↓

Apply evaluation function

↓

Back up values through the tree

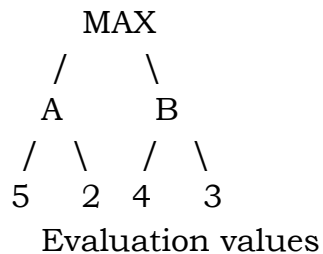
↓

Select the best root move

This is called **depth-limited Minimax**.

Simple example

Suppose the AI searches only two levels.



At the opponent level:

Value of A = $\min(5, 2) = 2$

Value of B = $\min(4, 3) = 3$

MAX chooses:

$\max(2, 3) = 3$

Therefore, the AI selects **B**.

But 5, 2, 4, 3 are estimated evaluation values, not actual win or loss values. Therefore, the result is an imperfect decision.

Iterative deepening

A very important real-time technique is **iterative deepening**.

The AI repeatedly searches with increasing depth:

Depth 1 search

↓

Depth 2 search

↓

Depth 3 search

↓

Depth 4 search

↓

Continue until time expires

After every completed search, the AI stores the best move.

Suppose the AI has five seconds:

Search Result

Depth 1 Completed

Depth 2 Completed

Depth 3 Completed

Depth 4 Completed

Depth 5 Started but not completed

When time expires, the AI uses the best move obtained from depth 4.

Why iterative deepening is useful

- A valid move is always available.
- The AI uses all available time.
- A deeper result replaces a shallower result.
- Earlier search results help order moves in later searches.

Horizon effect

The **horizon effect** occurs when an important event is just beyond the search limit.

Example:

Depth 1 → Depth 2 → Depth 3 → Depth 4 | Queen is lost at depth 5
 ↑
 Outside search horizon

At depth 4, the position may look good.

However, at depth 5, the AI may lose its queen. Since the AI stopped at depth 4, it cannot see the danger.

This is called the horizon effect.

Simple analogy

A student checks weather predictions only until Friday and plans an outdoor event on Saturday.

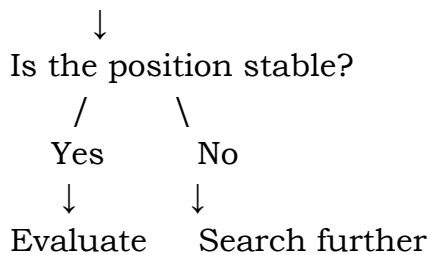
Rain occurs on Saturday, but it was beyond the period the student examined.

Quiescence search

Quiescence search reduces the horizon effect.

When the normal depth limit is reached, the AI checks whether the position is stable.

Depth limit reached



The AI may continue searching if:

- A major piece is about to be captured
- The king is under attack
- A forced move exists
- An important exchange is occurring

The AI stops only when the position becomes relatively quiet or stable.

Time management

A game-playing AI should not spend the same amount of time on every move.

It may spend more time when:

- The position is complex
- Many moves appear equally good
- The king is under threat
- The game is near a critical point

It may spend less time when:

- Only one legal move exists
- One move is obviously best
- The position is already known
- The move was calculated earlier

Therefore, real-time decision-making also involves deciding:

“How long should I think before making this move?”

Perfect decision versus imperfect real-time decision

Perfect decision

Searches the complete tree

Gives a guaranteed optimal move

Uses terminal utility values

Requires sufficient time

Suitable for small games

Imperfect real-time decision

Searches only part of the tree

Gives the best move found so far

Uses estimated evaluation values

Works under a time limit

Necessary for large games

Constraint Satisfaction Problems

1.A Constraint Satisfaction Problem, or **CSP**, is a problem where we must assign values to variables while satisfying given rules or restrictions.

Simple definition:

A Constraint Satisfaction Problem is a problem defined by variables, possible values, and constraints. The goal is to assign values to all variables without violating any constraint.

2. Basic Structure of CSP

A CSP has three main parts:

CSP = Variables + Domains + Constraints

Or formally:

CSP = (X, D, C)

Where:

Symbol Meaning

X Set of variables

Symbol Meaning

D Set of domains

C Set of constraints

3. Components of a CSP

3.1 Variables

Variables are the things for which we need to find values.

Example:

In map colouring:

Variables = A, B, C, D

Each variable represents a region in the map.

3.2 Domains

A domain is the set of possible values that a variable can take.

Example:

If the colours are:

Red, Green, Blue

Then:

Domain(A) = {Red, Green, Blue}

Domain(B) = {Red, Green, Blue}

Domain(C) = {Red, Green, Blue}

3.3 Constraints

Constraints are the rules that restrict which values can be assigned.

Example:

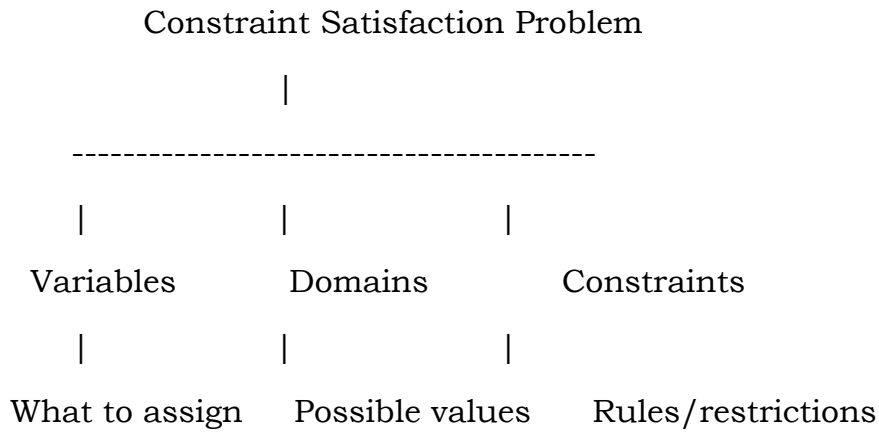
If region A is adjacent to region B, they should not have the same colour.

$A \neq B$

This means:

A and B must have different colours.

4. Simple CSP Diagram



5. Easy Example: Map Colouring Problem

Problem

Colour the map using three colours:

Red, Green, Blue

Condition:

Adjacent regions must not have the same colour.

Consider this map:

```

  A
 / \
B---C

```

Here:

- A is adjacent to B
- A is adjacent to C
- B is adjacent to C

CSP Formulation

Variables

A, B, C

Domains

$D(A) = \{\text{Red, Green, Blue}\}$

$D(B) = \{\text{Red, Green, Blue}\}$

$D(C) = \{\text{Red, Green, Blue}\}$

Constraints

$A \neq B$

$A \neq C$

$B \neq C$

One Valid Solution

$A = \text{Red}$

$B = \text{Green}$

$C = \text{Blue}$

Check:

$A \neq B \rightarrow \text{Red} \neq \text{Green}$

$A \neq C \rightarrow \text{Red} \neq \text{Blue}$

$B \neq C \rightarrow \text{Green} \neq \text{Blue}$

All constraints are satisfied.

Therefore, this is a valid CSP solution.

6. Constraint Graph

A CSP can be represented using a **constraint graph**.

In a constraint graph:

Element Meaning

Node Variable

Edge Constraint between variables

Example:

A

/ \
B---C

Here:

- A, B, and C are variables.
- Edges show constraints.
- The edge between A and B means $A \neq B$.

7. Important Terms in CSP

7.1 Assignment

An assignment means giving a value to a variable.

Example:

A = Red

7.2 Partial Assignment

A partial assignment means only some variables have values.

Example:

A = Red

B = Green

C = Not assigned

7.3 Complete Assignment

A complete assignment means every variable has a value.

Example:

A = Red

B = Green

C = Blue

7.4 Consistent Assignment

A consistent assignment does not violate any constraint.

Example:

$A = \text{Red}$

$B = \text{Green}$

If the constraint is:

$A \neq B$

then this assignment is consistent.

7.5 Solution

A solution is a **complete and consistent assignment**.

That means:

All variables have values

+

No constraint is violated

=

Solution

8. Types of Constraints

8.1 Unary Constraint

A unary constraint applies to one variable.

Example:

$A \neq \text{Red}$

Meaning:

Variable A cannot take Red.

Another example:

$\text{Age} \geq 18$

8.2 Binary Constraint

A binary constraint applies to two variables.

Example:

$A \neq B$

This is common in map colouring.

8.3 Global Constraint

A global constraint applies to many variables.

Example:

AllDifferent(A, B, C, D)

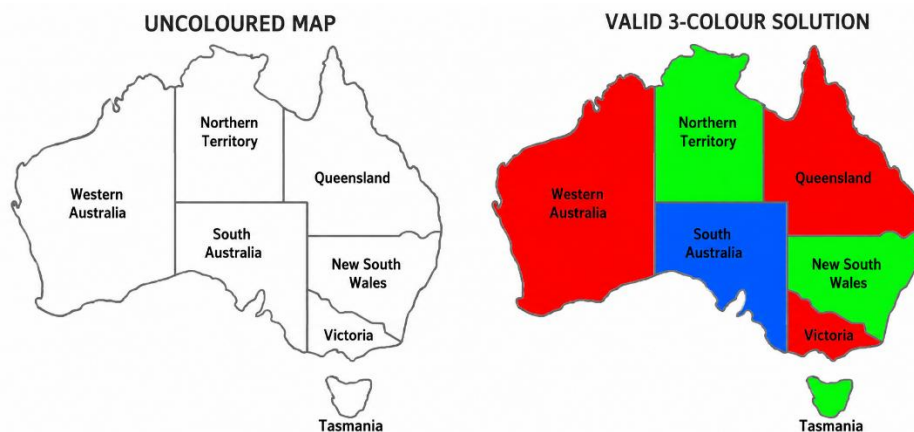
Meaning:

A, B, C, and D must all have different values.

Sudoku uses many global constraints.

CSP EXAMPLE:

GRAPH COLORING:



- Variables: WA, NT, Q, NSW, V, SA, T
- Domains: $D_i = \{\text{red, green, blue}\}$
- Constraints: adjacent regions must have different colors.
 - E.g. $WA \neq NT$ (if the language allows this)
 - E.g. $(WA, NT) \neq \{(\text{red, green}), (\text{red, blue}), (\text{green, red}), \dots\}$
- A state is defined as an assignment of values to some or all variables.
- Consistent assignment: assignment does not violate the constraints.
- A solution to a CSP is a complete assignment that satisfies all constraints.

Solution:

- {WA=red, NT=green, Q=red, NSW=green, V=red, SA=blue, T=green}
- Simple example of a formal representation language

CSP benefits

- Standard representation language
- Generic goal and successor functions
- Useful general-purpose algorithms with more power than standard search algorithms,
including generic heuristics.

Applications:

- Time table problems (exam/teaching schedules)
- Assignment problems (who teaches what)

Constraint Propagation in AI

1. Constraint propagation is a technique used in **Constraint Satisfaction Problems (CSPs)** to reduce the possible values of variables by applying constraints.

Simple definition:

Constraint propagation means using constraints to remove impossible values from variable domains.

It helps the AI system avoid unnecessary search.

2. Recall: CSP Components

A CSP has:

CSP = Variables + Domains + Constraints

Example:

Variables = A, B

Domains = {Red, Green, Blue}

Constraint = $A \neq B$

If $A = \text{Red}$, then B cannot be Red.

So we remove Red from B 's domain.

Before:

$A = \{\text{Red}\}$

$B = \{\text{Red, Green, Blue}\}$

After propagation:

$A = \{\text{Red}\}$

$B = \{\text{Green, Blue}\}$

This is **constraint propagation**.

3. Why Is It Called Propagation?

“Propagation” means **spreading**.

When one variable gets a value, its effect spreads to other connected variables.

$A = \text{Red}$

↓

$A \neq B$

↓

B cannot be Red

↓

Remove Red from B 's domain

If B 's domain becomes smaller, it may affect another variable C .

A affects B

B affects C

C affects D

So information spreads through the CSP.

4. Simple Diagram

A ----- B ----- C

Constraint:

$A \neq B$

$B \neq C$

Suppose:

A = Red

Then:

B cannot be Red

If later:

B = Green

Then:

C cannot be Green

Propagation flow:

A = Red

↓

B = {Green, Blue}

↓

If B = Green

↓

C = {Red, Blue}

5. Purpose of Constraint Propagation

Constraint propagation is used to:

1. Reduce search space
2. Remove impossible values

3. Detect failure early
4. Make backtracking faster
5. Find forced assignments

Without constraint propagation, the algorithm may try many useless combinations.

With constraint propagation, many wrong choices are removed early.

Local Consistency in Constraint Propagation

Local Consistency in CSP

Local consistency is a technique used in Constraint Satisfaction Problems to check whether assignments involving a small number of variables satisfy the constraints.

It helps remove impossible values from variable domains before or during search.

Local consistency examines small parts of a CSP instead of checking the entire problem at once.

Why it is used

Local consistency is used to:

- reduce variable domains,
- detect conflicts early,
- reduce backtracking,
- simplify the CSP,
- improve search efficiency.

Main types of local consistency

1. Node Consistency

Node consistency checks **one variable**.

A variable is node-consistent when every value in its domain satisfies all unary constraints.

Example:

Constraint:

Values 1 and 2 are removed.

So X is now node-consistent.

2. Arc Consistency

Arc consistency checks **two variables** connected by a binary constraint.

For the arc:

every value in must have at least one supporting value in .

Example:

Check X:

- is supported by or
- is supported by
- has no support

Therefore:

The AC-3 algorithm is commonly used to enforce arc consistency.

AC-3 Algorithm

AC-3 stands for **Arc Consistency Algorithm 3**. It is used in binary Constraint Satisfaction Problems to remove values that cannot satisfy constraints with neighbouring variables.

Core idea

For an arc:

$$X_i \rightarrow X_j$$

every value in $D(X_i)$ must have at least one compatible value in $D(X_j)$.

A compatible value is called a **supporting value**.

Simple example

Constraint:

$$X < Y$$

Domains:

$$D(X) = \{1, 2, 3\}$$

$$D(Y) = \{2, 3\}$$

Check values of X :

- $X = 1$ has support: $Y = 2$ or 3
- $X = 2$ has support: $Y = 3$
- $X = 3$ has no supporting value

Therefore, remove 3:

$$D(X) = \{1, 2\}$$

The arc $X \rightarrow Y$ is now consistent.

AC-3 Algorithm**AC-3 steps**

1. Add all directed arcs to a queue.
2. Remove one arc (X_i, X_j) .
3. Apply REVISE(X_i, X_j).
4. Remove unsupported values from $D(X_i)$.
5. If $D(X_i)$ becomes empty, return failure.
6. If $D(X_i)$ changes, add related arcs back to the queue.
7. Continue until the queue becomes empty.

REVISE(X_i, X_j)

1. revised $\leftarrow$ false
2. for each value x in Domain(X_i):
 - if no value y in Domain(X_j)
 - satisfies the constraint between X_i and X_j :
 - remove x from Domain(X_i)

revised $\leftarrow$ true

3. return revised

3. Path Consistency

Path consistency checks **three variables**.

A valid assignment to two variables must be extendable to a third variable.

Example:

Suppose:

There must be some value of Y satisfying:

Since:

works, the pair is path-consistent.

But for:

there is no Y such that:

So this pair is not path-consistent.

k-Consistency

A CSP is k -consistent if every consistent assignment to variables can be extended to any k -th variable.

Backtracking Search for CSPs

1. Meaning

Backtracking search is a method used to solve **Constraint Satisfaction Problems**, or **CSPs**.

It assigns values to variables one by one. After assigning each value, it checks whether the assignment satisfies the constraints.

If the assignment violates any constraint, the algorithm **goes back** and tries another value.

Backtracking means **try a value, check it, and go back when it fails**.

2. Recall: What is a CSP?

A Constraint Satisfaction Problem has three parts:

CSP = Variables + Domains + Constraints

Component Meaning

Variables Items to be assigned values

Domains Possible values for each variable

Constraints Rules that must be satisfied

3. Simple Example: Map Colouring

Suppose we have three regions:

A

/ \

B---C

Available colours:

{Red, Green, Blue}

Constraints:

$A \neq B$

$A \neq C$

$B \neq C$

This means adjacent regions should not have the same colour.

4. Backtracking Working Idea

Start with empty assignment

↓

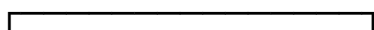
Select one unassigned variable

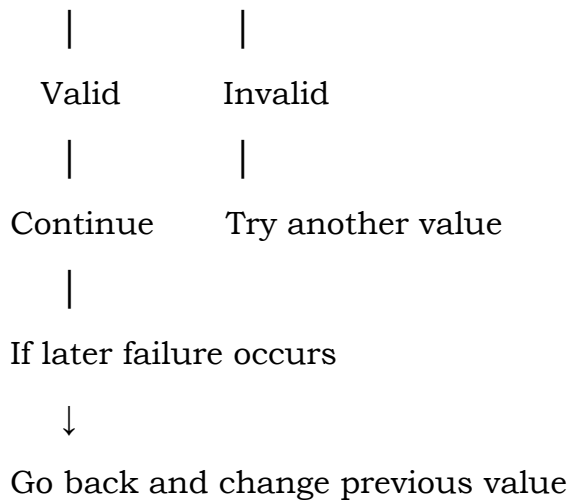
↓

Try one value from its domain

↓

Check constraints





5. Step-by-Step Example

Step 1: Assign A

Choose:

A = Red

Current assignment:

{A = Red}

This is valid because no constraint is violated.

Step 2: Assign B

Try:

B = Red

But:

A = Red

B = Red

This violates:

$A \neq B$

So reject B = Red.

Now try:

B = Green

Current assignment:

A = Red

B = Green

This is valid.

Step 3: Assign C

Try:

C = Red

This violates:

$A \neq C$

So reject it.

Try:

C = Green

This violates:

$B \neq C$

So reject it.

Try:

C = Blue

Now:

A = Red

B = Green

C = Blue

All constraints are satisfied.

Therefore, the solution is:

A = Red

B = Green

C = Blue

6. Search Tree Diagram

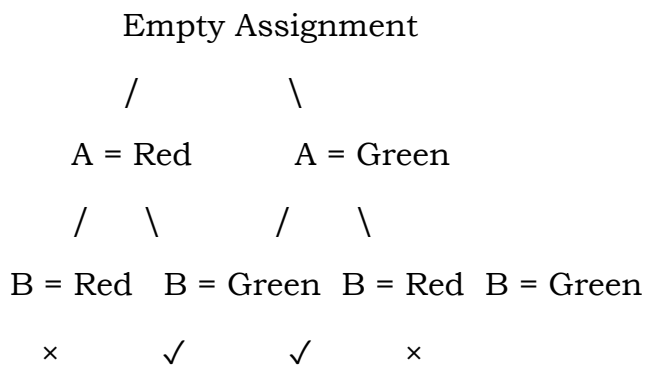
For two variables:

Variables: A, B

Domain: {Red, Green}

Constraint: $A \neq B$

Search tree:



Here:

× = Constraint violation

✓ = Valid solution

Backtracking avoids continuing with invalid branches.

7. Why is it Called Backtracking?

Suppose we reach this situation:

A = Red

B = Green

C = Blue

D = ?

Now assume no value works for D.

D = Red → conflict

D = Green → conflict

D = Blue → conflict

Then the algorithm goes back:

Remove C = Blue

Try another value for C

This backward movement is called **backtracking**.

8. Backtracking Algorithm

```

function BACKTRACKING-SEARCH(csp) returns a solution or failure
    return RECURSIVE-BACKTRACKING({}, csp)
function RECURSIVE-BACKTRACKING(assignment, csp)
    returns a solution or failure
    if assignment is complete then
        return assignment
    var ← SELECT-UNASSIGNED-VARIABLE(VARIABLES[csp], assignment,
    csp)
    for each value in ORDER-DOMAIN-VALUES(var, assignment, csp) do
        if value is consistent with assignment
            given CONSTRAINTS[csp] then add {var = value} to assignment
            result ← RECURSIVE-BACKTRACKING(assignment, csp)
            if result ≠ failure then
                return result
            remove {var = value} from assignment //Backtracking happens here
    return failure
  
```

9. Meaning of Important Terms

Assignment

Giving a value to a variable.

A = Red

Partial Assignment

Only some variables have values.

A = Red

B = Green

C = not assigned

Complete Assignment

All variables have values.

A = Red

B = Green

C = Blue

Consistent Assignment

An assignment that does not violate any constraint.

Example:

A = Red

B = Green

If constraint is:

$A \neq B$

then this assignment is consistent.

Solution

A solution is a **complete and consistent assignment**.

All variables assigned

+

No constraint violated

=

Solution

CRYPTARITHMETIC PROBLEM

- Cryptarithmic Problem is a type of **constraint satisfaction problem** where the game is about digits and its unique replacement either with alphabets or other symbols.
- In cryptarithmic problem, the digits (0–9) get substituted by some possible alphabets or symbols.
- The task in cryptarithmic problem is to substitute each digit with an alphabet to get the result arithmetically correct.

- We can perform all the arithmetic operations on a given cryptarithmic problem.

Constraints for cryptarithmic problem:

- Unique digit to be replaced with a unique alphabet (no repeated digits).
- The result should satisfy the predefined arithmetic rules, i.e., $2+2 = 4$.
- Digits should be from 0–9 only.
- In addition operation only one carry forward.
- The problem can be solved from both sides, i.e., left-hand side (L.H.S), or right-hand side (R.H.S).

Example 1:

Given a cryptarithmic problem, i.e.,

$$\text{SEND} + \text{MORE} = \text{MONEY}$$

```

S E N D
+ M O R E
-----
M O N E Y

```

Solution shown:

```

S E N D
9 5 6 7

+ M O R E
1 0 8 5
-----
M O N E Y
1 0 6 5 2

```

Therefore:

$$9567 + 1085 = 10652$$

Step 1:

Starting from the left-hand side (L.H.S), the terms are S and M. Assign a digit which could give a satisfactory result. Let's assign **S** → **9** and **M** → **1**.

$$\begin{array}{r}
 S \qquad 9 \\
 + M \qquad + 1 \\
 \hline
 MO \qquad 10
 \end{array}$$

Hence, we get a satisfactory result by adding up the terms and got an assignment for O as **O** → **0** as well.

Step 2:

Now, move ahead to the next terms **E** and **O** to get **N** as its output.

Step 2

$$\begin{array}{r}
 E \qquad 5 \\
 + O \qquad + 0 \\
 \hline
 N \qquad 5
 \end{array}$$

- Adding **E** and **O**, which means **5+0=0**, which is not possible because we cannot assign the same digit to two letters.
- Add **carry 1** to the value E to change the value of alphabet.

With carry:

$$1 \leftarrow \text{carry}$$

$$\begin{array}{r}
 E \qquad 5 \\
 + O \qquad + 0 \\
 \hline
 N \qquad 6
 \end{array}$$

Step 3

Further, adding the next two terms N and R we get,

Initially:

$$\begin{array}{r}
 N \qquad 6 \\
 + R \qquad + 8 \\
 \hline
 E \qquad 14
 \end{array}$$

With carry:

$$1 \leftarrow \text{carry}$$

$$\begin{array}{r}
 N \qquad 6 \\
 + R \qquad + 8 \\
 \hline
 E \qquad 15
 \end{array}$$

But, we have already assigned **E** → **5**. Not possible with 5 to E. Again, after solving the whole problem, we will get a carryover on this term, so our answer will be satisfied.

Step 4

Again, on adding the last two terms, i.e., the rightmost terms D and E, we get Y as its result.

$$\begin{array}{r}
 D \qquad 7 \\
 + E \qquad + 5 \\
 \hline
 Y \qquad 12
 \end{array}$$

where 1 will be carry forward to the above term

So:

- **D = 7**
- **E = 5**
- **Y = 2**
- **Carry = 1**

Final values shown across the example

$$S = 9$$

$$E = 5$$

$$N = 6$$

$$D = 7$$

$$M = 1$$

$$O = 0$$

$$R = 8$$

$$Y = 2$$

Therefore:

$$SEND = 9567$$

$$MORE = 1085$$

$$MONEY = 10652$$

Note: The image says “5+0=0” in Step 2, but from the diagram and arithmetic it should be **5+0=5**.

Note: Refer Class room problems based on Graph colouring and map colouring.

Local Search for CSPs

- Local search algorithms turn out to be effective in solving many CSPs. They use a complete-state formulation: the initial state assigns a value to every variable, and the search changes the value of one variable at a time.
- For example, in the 8-queens problem (see Figure 4.3), the initial state might be a random configuration of 8 queens in 8 columns, and each step moves a single queen to a new position in its column.
- Typically, the initial guess violates several constraints. The point of local search is to eliminate the violated constraints. In choosing a new value for a variable, the most obvious heuristic is to select the value

that results in the minimum number of conflicts with other variables—the **min-conflicts** heuristic. The algorithm is shown in Figure 6.8 and its application to an 8-queens problem is diagrammed in Figure 6.9

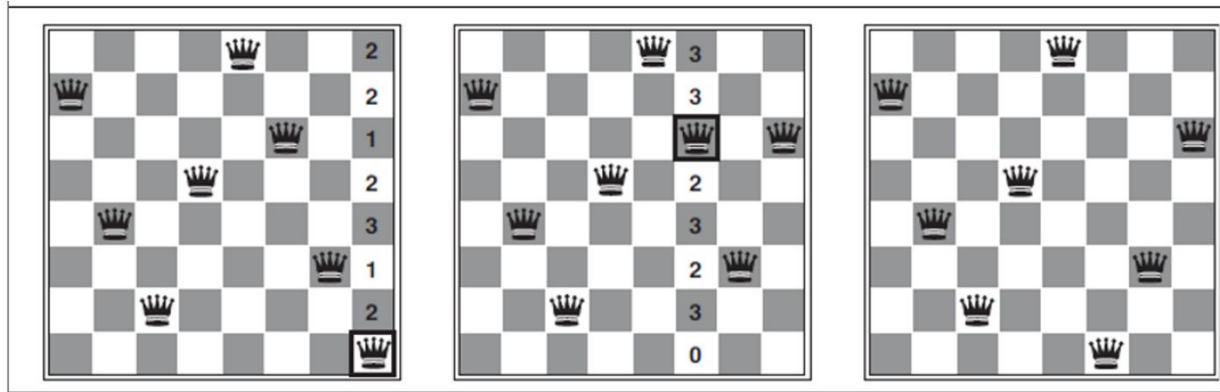


Figure 6.9 A two-step solution using min-conflicts for an 8-queens problem. At each stage, a queen is chosen for reassignment in its column. The number of conflicts (in this case, the number of attacking queens) is shown in each square. The algorithm moves the queen to the min-conflicts square, breaking ties randomly.

Algorithm for Local Search for CSPs

function **MIN-CONFLICTS**(csp, max_steps) returns a solution or failure

inputs:

csp, a constraint satisfaction problem

max_steps, the number of steps allowed before giving up

current ← an initial complete assignment for csp

for i = 1 to max_steps do

if current is a solution for csp then

return current

var ← a randomly chosen conflicted variable from csp.VARIABLES

value ← the value v for var that minimizes

CONFLICTS(var, v, current, csp)

set var = value in current

return failure

Important Characteristics

It uses a complete-state formulation

All variables are assigned from the beginning. The algorithm repairs incorrect assignments rather than constructing a solution gradually.

It is a local-search algorithm

It considers changes to the current assignment rather than exploring a complete search tree.

It is not always guaranteed to succeed

The algorithm can become stuck in a local minimum or cycle among assignments. A step limit and random restarts are commonly used to handle this.

It works well for large CSPs

Min-conflicts is especially effective when a problem has many possible solutions or when a nearly correct assignment can be repaired quickly.

The Structure of Problems

The structure of a **Constraint Satisfaction Problem** is represented using a **constraint graph**:

- Nodes represent variables.
- Edges represent constraints between variables.

When the constraint graph forms a tree, the CSP can be solved efficiently without backtracking.

1. Solving a Tree-Structured CSP

A **tree-structured CSP** has a constraint graph that is connected and contains no cycles.

Step 1: Select a root

Choose any variable as the root of the tree.

Example:

```

    A
   / \
  B  C

```

$$\begin{array}{c} / \quad \backslash \\ D \quad E \end{array}$$

Here, let A be the root.

Step 2: Topological ordering

Arrange the variables so that every parent appears before its children.

One possible ordering is:

A, B, C, D, E

This is called a **topological ordering**.

Another valid ordering is:

A, C, B, E, D

The essential condition is:

A child variable must always appear after its parent.

Step 3: Make the tree arc-consistent

Process the variables from the leaves toward the root.

For the ordering:

A, B, C, D, E

process in reverse:

$E \rightarrow B$

$D \rightarrow B$

$C \rightarrow A$

$B \rightarrow A$

For each parent-child pair, remove any value from the parent's domain that has no compatible value in the child's domain.

Example

Suppose:

$\text{Domain}(A) = \{1, 2, 3\}$

$\text{Domain}(B) = \{1, 2\}$

Constraint: $A < B$

Check each value of A:

- A = 1 has support because B = 2.
- A = 2 has no value of B greater than 2.
- A = 3 has no value of B greater than 3.

Therefore:

$\text{Domain}(A) = \{1\}$

Step 4: Assign values from root to leaves

After arc consistency is completed:

1. Choose any remaining value for the root.
2. Move from the root toward the leaves.
3. Assign each child a value consistent with its parent.

Because arc consistency was already established, a suitable child value will always be available.

Converting a General Constraint Graph into a Tree

The correct statement is:

There are two main ways to reduce a cyclic constraint graph into a tree-like structure.

They are:

1. Cutset conditioning
2. Tree decomposition

2. Cutset Conditioning

A **cycle cutset** is a group of variables whose removal makes the remaining constraint graph a tree.

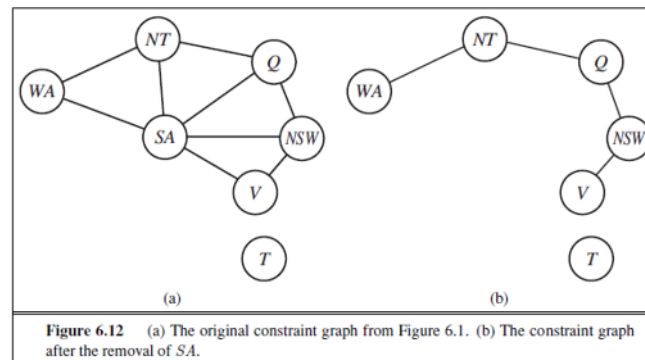


Figure 6.12 (a) The original constraint graph from Figure 6.1. (b) The constraint graph after the removal of SA.

Example

Consider the cyclic graph:

A — B

| |

D — C

It contains the cycle:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow A$

Remove variable A:

B — C — D

The remaining graph is a tree.

Therefore:

$S = \{A\}$

is a cycle cutset.

Cutset Conditioning Algorithm

Step 1: Select the cutset

Choose a set of variables S such that removing S makes the remaining constraint graph a tree.

Step 2: Try possible assignments

For every possible assignment to the variables in S:

- Check whether the assignment satisfies all constraints among the variables in S.

- Remove inconsistent values from the domains of the remaining variables.
- Solve the remaining tree-structured CSP.

Step 3: Return the solution

When the remaining tree CSP has a solution, combine it with the assignment made to S.

3. Tree Decomposition

In **tree decomposition**, the original CSP is divided into smaller groups of variables called:

- Subproblems
- Clusters
- Bags

These clusters are then connected in the form of a tree.

Example

Suppose the original graph is:

A — B

| / |

D — C

A possible tree decomposition is:

{A, B, C} — {A, C, D}

The two clusters share the variables:

A and C

Their assignments for A and C must agree.

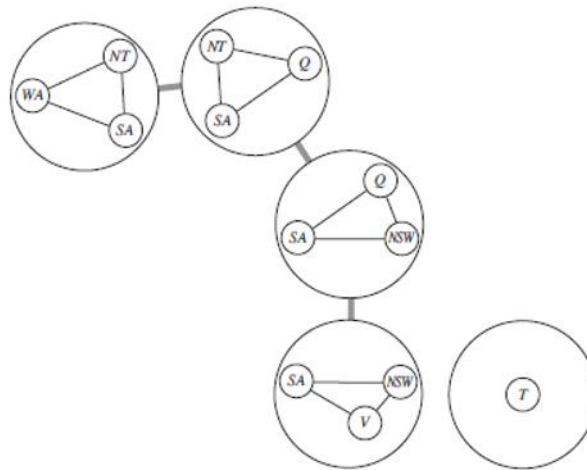


Figure 6.13 A tree decomposition of the constraint graph in Figure 6.12(a).

Requirements of Tree Decomposition

A valid tree decomposition must satisfy three conditions.

1. Every variable must be included

Every variable in the original CSP must appear in at least one cluster.

For example:

Cluster 1 = {A, B, C}

Cluster 2 = {A, C, D}

All variables A, B, C, D are included.

2. Every constraint must be included

When two variables are connected by a constraint in the original problem, they must appear together in at least one cluster.

For example, when the original problem contains:

Constraint between B and C

there must be a cluster containing both:

{A, B, C}

3. Running-intersection property

When a variable appears in two clusters, it must also appear in every cluster along the path between them.

Example:

$\{A, B\} \text{ --- } \{A, C\} \text{ --- } \{A, D\}$

Variable A appears in the first and third clusters. Therefore, it must also appear in the middle cluster.

This condition ensures that all clusters use a consistent value for A.

Knowledge-Based Agents

Knowledge Representation in Artificial Intelligence

1. Introduction

Knowledge Representation, or **KR**, is the method used to store real-world knowledge in a form that a computer or intelligent agent can understand and use.

Knowledge representation converts human knowledge into a structured form so that an AI system can reason, make decisions and solve problems.

Simple example

A human knows:

All birds have wings.

A parrot is a bird.

An AI system can represent this as:

$\text{Bird}(x) \rightarrow \text{HasWings}(x)$

$\text{Bird}(\text{Parrot})$

From this knowledge, the system can infer:

$\text{HasWings}(\text{Parrot})$

2. What is Knowledge?

In AI, **knowledge** means information about:

- Objects
- Events
- People
- Places
- Properties

- Relationships
- Rules
- Actions

Example

Hyderabad is a city.

A dog is an animal.

Rain makes roads wet.

A patient with fever may have an infection.

These statements describe facts and relationships in the real world.

3. Definition

Knowledge representation is the process of organizing and encoding knowledge so that an artificial intelligence system can store it, retrieve it and reason with it.

It acts as a connection between:

Human knowledge



Knowledge representation



Computer reasoning



Decision or action

4. Why is Knowledge Representation Needed?

An intelligent system requires knowledge representation to:

- Understand the environment
- Store facts and rules
- Derive new information
- Answer questions

- Solve problems
- Make decisions
- Explain its conclusions
- Plan future actions

Example

A medical AI system may store:

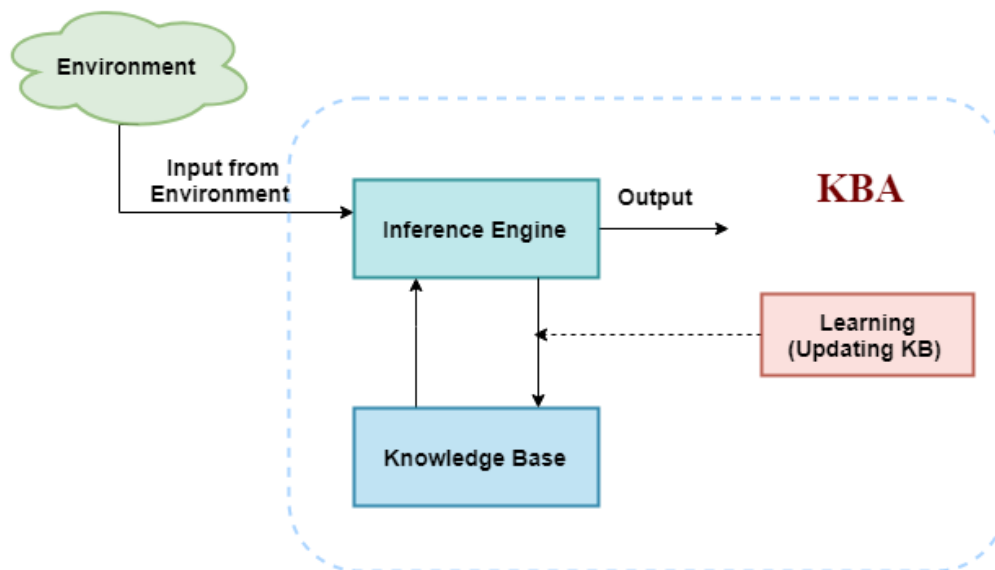
Fever is a symptom.

Cough is a symptom.

Fever and cough may indicate an infection.

When a patient has fever and cough, the system can reason about a possible illness.

Architecture of Knowledge-Based Agents



First, the knowledge-based agent receives the data from environment through **sensors** then it retrieves the relevant information, rules, facts from the **knowledge base** to understand the context. It then processes the relevant information through **inference system** using techniques like forward or backward chaining. Based on this reasoning, **agent** decides the best action.

Once the decision is made, the agent executes the action using **actuators** to affect the environment. If the agent is capable of learning, it updates the new information to the knowledge base for future tasks.

What is Knowledge base?

Knowledge base is the critical component of the Knowledge-Based Agent, which includes all the **information**, **rules**, and **facts** necessary for reasoning and decision.

It gives the agent all the information it needs to reason, make informed judgments, and act appropriately with reference to its environment.

- **Facts** are information about the world. For example, "The Earth orbits the Sun once every 365.25 days".
- **Rules** are logical statements that relate facts. For example, "If the temperature drops below 0C, water will freeze into ice".
- **Updates:** The knowledge base can be created or updated during the process in which an agent learns or introduces new knowledge so that over a period, its decision-making skill is increased.

For example, **smart thermostat** saves information as "Room temperature needs to be at 22C", "If the temperature of room has exceeded to more than 25C then turn on air conditioner".

Inference System

The inference engine proceeds to **process** knowledge in the knowledge base using logical reasoning to help the agent take intelligent decision and solve the problem. Imagine this as a tool to determine "what is next" from facts and regulations.

*The inference system acts like the **thinking** part of the agent. It begins with a set of given facts and uses reasoning to obtain new facts or decide what to do.*

This system also creates **new knowledge** from the existing information and includes them in the knowledge base to assist future decision making.

- In practical, an inference system is constructed around algorithms and logical representations such as propositional or first-order logic.
- Inference systems are used in many applications, such as expert systems, natural language processing, and automated reasoning. This allows machines to think logically and make judgments like humans.

This technique uses two basic approaches to introduce logic –

- **Forward Chaining:** This is based on the assumptions given. The inference engine checks the rules, applies them, and keep inferring the new facts until it reaches a conclusion or goal.

- **Backward Chaining:** Start from the goal and work backward. It determines which facts or conditions need to be satisfied to achieve that goal.

For example, the thermostat checks the current temperature, applies the criteria, and concludes, "The room is 27C, so I need to activate the cooling system." This decision is made by inference system.

Sensors and Actuators

In knowledge-based agent **sensors** collect environmental information such as temperature, light, and movement, while **actuators** use that information to perform actions such as moving, turning, or cleaning. Robot vacuums use sensors to look for dirt and actuators to clean it or to avoid obstacles.

Operations Performed by KBA

Knowledge-based agents utilize three key procedures to take intelligent actions –

- **TELL:** The agent updates its knowledge base with new information from the environment. For example, if a robot sees a door open, it tells the knowledge base, "The door is open."
- **ASK:** The agent asks the knowledge base what decision it should take. For example, it may ask, "Should I close the door or move forward?"
- **PERFORM:** The agent will follow the guidance of the knowledge base, such as closing the door or moving forward. It performs the selected option.

These processes enable the agent to learn, make smart judgments, and efficiently adapt to its surroundings.

A Simple Knowledge Based Agent

Knowledge-Based Agent (KBA) processes perceptions, reasons using its knowledge base, and makes decisions. The following pseudocode represents a basic Knowledge-Based Agent –

function KB-AGENT(percept):

 persistent: KB, a knowledge base

 t, a counter, initially 0, indicating time

 TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))

 Action = ASK(KB, MAKE-ACTION-QUERY(t))

 TELL(KB, MAKE-ACTION-SENTENCE(action, t))

$t = t + 1$

return action

How a Knowledge-Based Agents works

A Knowledge-Based Agent continuously perceives its environment, utilizes stored knowledge for reasoning, and makes informed decisions. Below is the explanation of how it operates –

- **MAKE-PERCEPT-SENTENCE:** The agent updates its knowledge base with whatever it observes(environment) at time t by converting the percept into a logical sentence.
- **MAKE-ACTION-QUERY:** The agent looks in its knowledge base to determine which action it should execute given the current time step and latest knowledge.
- **MAKE-ACTION-SENTENCE:** After choosing an action the agent updates its knowledge base with a logical statement that describes the action which the agent intends to execute.
- The function **returns** the chosen action, which the agent will carry out in the environment.

Types of Logical Representations

They are two types of logical representation which are mentioned below –

1.Propositional logic (PL) has basic statements (propositions) that logical operators like AND, OR, and NOT connect.

2.First-Order Logic (FOL) expands on PL by adding objects, relationships, and quantifiers such as (for all) and (exists).

Propositional Logic in Artificial Intelligence

Propositional logic is used by artificial intelligence to allow a computer to express propositions concerning a particular subject in formally logical ways. It combines propositions (these are statements that must be either true or false) with logical connectives such as $\wedge$, $\vee$ and $\neg$. Many automated logic, knowledge representation and decision-making systems are based on this logic that can be provided.

Given the formal definition of the situations, the so-called facts, which are represented by the so-called propositions, propositional logic provides for an

organized approach to the reasoning about situations and, besides, makes the AI system able to conclude new facts based on the available ones.

While less fully formed than predicate logic, due to the limited number of available expressions, first-order logic has a central place in making intelligent agents. It defines a technique of knowledge representation in logical and mathematical form.

Syntax of propositional Logic:

The syntax of propositional logic defines the allowable sentences for the knowledge representation. There are two types of Propositions:

- Atomic Propositions
- Compound propositions

Atomic Proposition: Atomic propositions are simple propositions. It consists of a single proposition symbol. These are the sentences which must be either true or false.

Example 1:

1. $2+2$ is 4; it is an atomic proposition as it is a fact.
2. "The Sun is cold" is also a proposition as it is a false fact.

Compound proposition: Compound propositions are constructed by combining simpler or atomic propositions, using parenthesis and logical connectives.

Example 2:

1. "It is raining today, and the street is wet."
2. "Ankit is a doctor, and his clinic is in Mumbai."

Logical Connectives

Logical connectives are used to connect two simpler propositions or represent a sentence logically. We can create compound propositions with the help of logical connectives. There are mainly five connectives, which are given as follows:

- **Negation:** A sentence such as $\neg P$ is called negation of P. A literal can be either Positive literal or negative literal.
- **Conjunction:** A sentence that has $\wedge$ a connective, such as $P \wedge Q$, is called a conjunction.

Example: Rohan is intelligent and hardworking. It can be written as,

P= Rohan is intelligent,

Q= Rohan is hardworking. $\rightarrow P \wedge Q$.

- **Disjunction:** A sentence that has $\vee$ a connective, such as $P \vee$, is called disjunction, where P and Q are the propositions.

Example: "Ritika is a doctor or Engineer"

Here P= Ritika is Doctor. Q= Ritika is Doctor, so we can write it as $P \vee Q$.

- **Implication:** A sentence such as $P \rightarrow Q$ is called an implication. Implications are also known as if-then rules. It can be represented as

If it is raining, then the street is wet.

Let P= It is raining, and Q= Street is wet, so it is represented as $P \rightarrow Q$

- **Biconditional:** A sentence such as $P \leftrightarrow Q$ is a Biconditional sentence example: If I am breathing, then I am alive

P= I am breathing, Q= I am alive, it can be represented as $P \leftrightarrow Q$.

Following is the summarized table for Propositional Logic Connectives:

Connective symbols	Word	Technical Term	Example
$\wedge$	AND	Conjunction	$A \wedge B$
$\vee$	OR	Disjunction	$A \vee B$
$\rightarrow$	Implies	Implication	$A \rightarrow B$
$\leftrightarrow$	If and only if	Biconditional	$A \leftrightarrow B$
$\neg$ or $\sim$	Not	Negation	$\neg A$ or $\neg B$

 Tpoint Tech

Truth Table

In propositional logic, we need to know the truth values of propositions in all possible scenarios. We can combine all the possible combinations with logical connectives, and the representation of these combinations in a tabular format is called a Truth table. Following is the truth table for all logical connectives:

Rules of Inference in Artificial intelligence

Inference:

In artificial intelligence, we need intelligent computers which can create new logic from old logic or by evidence, **so generating the conclusions from evidence and facts is termed as Inference.**

Inference rules:

Inference rules are the templates for generating valid arguments. Inference rules are applied to derive proofs in artificial intelligence, and the proof is a sequence of the conclusion that leads to the desired goal.

In inference rules, the implication among all the connectives plays an important role. Following are some terminologies related to inference rules:

- **Implication:** It is one of the logical connectives which can be represented as $P \rightarrow Q$. It is a Boolean expression.
- **Converse:** The converse of implication, which means the right-hand side proposition goes to the left-hand side and vice-versa. It can be written as $Q \rightarrow P$.
- **Contrapositive:** The negation of converse is termed as contrapositive, and it can be represented as $\neg Q \rightarrow \neg P$.
- **Inverse:** The negation of implication is called inverse. It can be represented as $\neg P \rightarrow \neg Q$.

From the above term some of the compound statements are equivalent to each other, which we can prove using truth table:

P	Q	$P \rightarrow Q$	$Q \rightarrow P$	$\neg Q \rightarrow \neg P$	$\neg P \rightarrow \neg Q$
T	T	T	T	T	T
T	F	F	T	F	T
F	T	T	F	T	F
F	F	T	T	T	T

Hence from the above truth table, we can prove that $P \rightarrow Q$ is equivalent to $\neg Q \rightarrow \neg P$, and $Q \rightarrow P$ is equivalent to $\neg P \rightarrow \neg Q$.

Types of Inference rules:

1. Modus Ponens:

The Modus Ponens rule is one of the most important rules of inference, and it states that if P and $P \rightarrow Q$ is true, then we can infer that Q will be true. It can be represented as:

Notation for Modus ponens:
$$\frac{P \rightarrow Q, P}{\therefore Q}$$

Example:

Statement-1: "If I am sleepy then I go to bed" $\Rightarrow P \rightarrow Q$

Statement-2: "I am sleepy" $\Rightarrow P$

Conclusion: "I go to bed." $\Rightarrow Q$.

Hence, we can say that, if $P \rightarrow Q$ is true and P is true then Q will be true.

Proof by Truth table:

P	Q	$P \rightarrow Q$
0	0	0
0	1	1
1	0	0
1	1	1

2. Modus Tollens:

The Modus Tollens rule state that if $P \rightarrow Q$ is true and $\neg Q$ is true, then $\neg P$ will also true. It can be represented as:

Notation for Modus Tollens:
$$\frac{P \rightarrow Q, \neg Q}{\neg P}$$

Statement-1: "If I am sleepy then I go to bed" $\Rightarrow P \rightarrow Q$

Statement-2: "I do not go to the bed." $\Rightarrow \neg Q$

Statement-3: Which infers that "**I am not sleepy**" $\Rightarrow \neg P$

Proof by Truth table:

P	Q	$\neg P$	$\neg Q$	$P \rightarrow Q$
0	0	1	1	1
0	1	1	0	1
1	0	0	1	0
1	1	0	0	1

3. Hypothetical Syllogism:

The Hypothetical Syllogism rule state that if $P \rightarrow R$ is true whenever $P \rightarrow Q$ is true, and $Q \rightarrow R$ is true. It can be represented as the following notation:

Example:

Statement-1: If you have my home key then you can unlock my home. $P \rightarrow Q$

Statement-2: If you can unlock my home then you can take my money. $Q \rightarrow R$

Conclusion: If you have my home key then you can take my money. $P \rightarrow R$

Proof by truth table:

P	Q	R	$P \rightarrow Q$	$Q \rightarrow R$	$P \rightarrow R$
0	0	0	1	1	1
0	0	1	1	1	1
0	1	0	1	0	1
0	1	1	1	1	1
1	0	0	0	1	1
1	0	1	0	1	1
1	1	0	1	0	0
1	1	1	1	1	1

4. Unit Resolution

The Disjunctive syllogism rule state that if $P \vee Q$ is true, and $\neg P$ is true, then Q will be true. It can be represented as:

Notation of Disjunctive syllogism: $\frac{P \vee Q, \neg P}{Q}$

Example:

Statement-1: Today is Sunday or Monday. $\implies P \vee Q$

Statement-2: Today is not Sunday. $\implies \neg P$

Conclusion: Today is Monday. $\implies Q$

Proof by truth-table:

P	Q	$\neg P$	$P \vee Q$
0	0	1	0
0	1	1	1
1	0	0	1
1	1	0	1

5. OR Introduction

The Addition rule is one the common inference rule, and it states that If P is true, then $P \vee Q$ will be true.

Notation of Addition: $\frac{P}{P \vee Q}$

Example:

Statement: I have a vanilla ice-cream. $\implies P$

Statement-2: I have Chocolate ice-cream.

Conclusion: I have vanilla or chocolate ice-cream. $\implies (P \vee Q)$

Proof by Truth-Table:

P	Q	$P \vee Q$
0	0	0
1	0	1
0	1	1
1	1	1

6. AND Elimination

The simplification rule state that if $P \wedge Q$ is true, then Q or P will also be true. It can be represented as:

Notation of Simplification rule: $\frac{P \wedge Q}{Q}$ Or $\frac{P \wedge Q}{P}$

Proof by Truth-Table:

P	Q	$P \wedge Q$
0	0	0
1	0	0
0	1	0
1	1	1

7. Resolution

Resolution is a rule of inference used to derive a new clause from two clauses that contain a pair of **complementary literals**.

General form

P

Therefore:

$P \vee Q$

Similarly:

Q

Therefore:

$P \vee Q$

8. AND Introduction

AND Introduction is a rule of inference used when two statements are both known to be true.

If P is true and Q is true, then we can conclude $P \wedge Q$.

General form

P

Q

Therefore: $= P \wedge Q$

For Negation :

P	$\neg P$
True	False
False	True

For Conjunction:

P	Q	$P \wedge Q$
True	True	True
True	False	False
False	True	False
False	False	False

For disjunction:

P	Q	$P \vee Q$
True	True	True
False	True	True
True	False	True
False	False	False

For Implication:

P	Q	$P \rightarrow Q$
True	True	True
True	False	False
False	True	True
False	False	True

Tpoint Tech

For Biconditional :

P	Q	$P \leftrightarrow Q$
True	True	True
True	False	False
False	True	False
False	False	True

Tpoint Tech

Precedence of connectives:

Just like arithmetic operators, there is a precedence order for propositional connectors or logical operators. This order should be followed while evaluating a propositional problem. Following is the list of the precedence order for operators:

Precedence	Operators
First Precedence	Parenthesis
Second Precedence	Negation
Third Precedence	Conjunction(AND)
Fourth Precedence	Disjunction(OR)
Fifth Precedence	Implication
Six Precedence	Biconditional

Logical Equivalence

Propositional logic is one of the features that have logical equivalence. The definition of logical equivalence is given by saying two propositions are logically equivalent if and only if the columns of a truth table are the same.

Assuming two propositions, A and B, we'll mark it by $A \Leftrightarrow B$, and this is the logical equivalence. From the below truth table, we see that the columns of $\neg A \vee B$ and $B \rightarrow A$ are identical, so A is Equivalent to B.

A	B	$\neg A$	$\neg A \vee B$	$A \rightarrow B$
T	T	F	T	T
T	F	F	F	F
F	T	T	T	T
F	F	T	T	T

Properties of Operators

Commutativity:

$P \wedge Q = Q \wedge P$, or

$P \vee Q = Q \vee P$.

Associativity:

$$(P \wedge Q) \wedge R = P \wedge (Q \wedge R),$$

$$(P \vee Q) \vee R = P \vee (Q \vee R)$$

Identity element:

$$P \wedge \text{True} = P,$$

$$P \vee \text{True} = \text{True}.$$

Distributive:

$$P \wedge (Q \vee R) = (P \wedge Q) \vee (P \wedge R).$$

$$P \vee (Q \wedge R) = (P \vee Q) \wedge (P \vee R).$$

DE Morgan's Law:

$$\neg (P \wedge Q) = (\neg P) \vee (\neg Q)$$

$$\neg (P \vee Q) = (\neg P) \wedge (\neg Q).$$

Double-negation elimination:

$$\neg (\neg P) = P.$$

Applications of Propositional Logic in AI

- **Knowledge Representation:** Expressed in the propositional logic, knowledge is presented formally in a certain structure. It enables them to store and process information or facts about the world. For instance, in knowledge-based systems, the knowledge base comprises propositions and logical rules.
- **Problem Solving and Planning:** This serves the purpose of allowing AI planners to solve problems and create action sequences given particular goals. For instance, the STRIPS planning system assists propositional logic in the following aspects: it is used in representing preconditions and effects of actions.
- **Decision Making:** If applied properly, it assists in considering top choices and selecting the best approach to be used. Ername logical rules for decision criteria or even truth tables can be implemented in order to evaluate the performance of various choices.
- **Natural Language Processing (NLP):** However, this is also used in [NLP](#) applications such as semantic parsing, which involves turning natural language sentences into logical forms. This assists in the

meaning post-processing of a sentence and enables one to give reasons for the meaning of a given sentence.

Some Basic Facts about Propositional Logic

- Propositional logic is also called [Boolean](#) logic, as it works on 0 and 1.
- In propositional logic, we use symbolic variables to represent the logic, and we can use any symbol to represent a proposition, such as A, B, C, P, Q, R, etc.
- Propositions can be either true or false, but they cannot be both.
- Propositional logic consists of an object, relations or functions, and logical connectives.
- These connectives are also called logical operators.
- Propositions and connectives are the basic elements of propositional logic.
- Connectives can be said as a logical operator which connects two sentences.
- A proposition formula that is always true is called tautology, and it is also called a valid sentence.
- A proposition formula that is always false is called a Contradiction.
- A proposition formula which has both true and false values is called.
- Statements that are questions, commands, or opinions, such as "Where is Rohini?", "How are you?" and "What is your name?" are not propositions.

Limitations of Propositional Logic

Although it has numerous advantages, it also has some drawbacks, and they are as follows:

- **Lack of Expressiveness:** It cannot differentiate scenarios such as 'All humans are mortal'.
- **Scalability:** The Excel table goes up with the number of propositions as the number of rows as rows in the [Excel](#) table will increase.
- **Limited Inference:** It only considers and works with true and false propositions and cannot handle the probabilities.
- **No Quantifiers:** Unlike predicate logic, it does not cover the use of quantifiers for all symbols $\forall$ and their existing symbols $\exists$.

- **Inability to Handle Uncertainty:** It cannot accommodate probabilities or partial truths to assist, which makes it deficient in uncertain conditions.
- **Lack of Context Awareness:** It removes the meaning or context of statements, which in turn reduces the ability to decipher complex situations.
- We cannot represent relations like ALL, some, or none with propositional logic.

Propositional Theorem Proving

Propositional theorem proving is the process of proving that a required conclusion logically follows from the facts and rules stored in a knowledge base.

In simple words:

The AI system uses known facts and logical rules to determine whether a statement can be proved.

Inference and Proofs

1. What is Inference?

Inference is the process of deriving new knowledge from facts and rules already stored in a knowledge base.

Inference tells an AI system how to move from known information to a new conclusion.

2. What is a Proof?

A **proof** is a sequence of valid inference steps showing how a conclusion is obtained from the given premises.

Inference is one reasoning step, whereas a proof may contain several inference steps.

1. Prove A Using Inference Rules

Given:

KB1: $((A \vee B) \wedge C) \rightarrow E$

KB2: $(D \wedge F) \rightarrow A$

KB3: $\neg E \vee D$

KB4: $B \wedge C$

KB5: F

Goal: A

Solution:

1. $B \wedge C$ — Given (KB4)
2. B — 1, And-Elimination
3. C — 1, And-Elimination
4. $A \vee B$ — 2, Or-Introduction
5. $(A \vee B) \wedge C$ — 3, 4, And-Introduction
6. E — 5, KB1, Modus Ponens
7. $\neg E \vee D$ — Given (KB3)
8. D — 6, 7, Unit Resolution
9. F — Given (KB5)
10. $D \wedge F$ — 8, 9, And-Introduction
11. A — 10, KB2, Modus Ponens

Therefore, A is proved.

2. Prove $\neg P_{1,2}$ Using Inference Rules

Given:

R1: $\neg P_{1,1}$

R2: $B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$

R3: $B_{2,1} \leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$

R4: $\neg B_{1,1}$

R5: $B_{2,1}$

Goal: $\neg P_{1,2}$

Solution

1. $B_{1,1} \leftrightarrow (P_{1,2} \vee P_{2,1})$ — Given (R2)
2. $(P_{1,2} \vee P_{2,1}) \rightarrow B_{1,1}$ — Biconditional Elimination
3. $\neg B_{1,1}$ — Given (R4)
4. $\neg(P_{1,2} \vee P_{2,1})$ — Modus Tollens (2, 3)

5. $\neg P_{1,2} \wedge \neg P_{2,1}$ — De Morgan's Law

6. $\neg P_{1,2}$ — And-Elimination

Therefore, $\neg P_{1,2}$ is proved.

3. Prove $A \wedge B$ Using Inference Rules

Given:

KB1: $(A \vee \neg C) \rightarrow B$

KB2: $\neg C \vee D$

KB3: $D \rightarrow A$

KB4: C

Goal: $A \wedge B$

Solution

1. C — Given (KB4)
2. $\neg C \vee D$ — Given (KB2)
3. D — From 1 and 2, **Unit Resolution / Disjunctive Syllogism**
4. $D \rightarrow A$ — Given (KB3)
5. A — From 3 and 4, **Modus Ponens**
6. $A \vee \neg C$ — From 5, **Or-Introduction**
7. $(A \vee \neg C) \rightarrow B$ — Given (KB1)
8. B — From 6 and 7, **Modus Ponens**
9. $A \wedge B$ — From 5 and 8, **And-Introduction**

Therefore,

$A \wedge B$ is proved.

Proof by Resolution

Proof by resolution is a method of proving a statement by showing that its **negation leads to a contradiction**.

It is also called **resolution refutation**.

Procedure

1. Convert all statements in the knowledge base into **CNF / clause form**.

2. Negate the goal to be proved.
3. Add the negated goal to the knowledge base.
4. Apply the **resolution rule** repeatedly.
5. If the **empty clause** is obtained, the goal is proved.

Resolution Refutation Example

Given:

KB1: $((A \vee B) \wedge C) \rightarrow E$

KB2: $D \wedge F \rightarrow A$

KB3: $\neg E \vee D$

KB4: $B \wedge C$

KB5: F

Goal: Prove A

Convert to Clause Form

KB1 gives:

KB1.1: $\neg A \vee \neg C \vee E$

KB1.2: $\neg B \vee \neg C \vee E$

KB2:

KB2: $\neg D \vee \neg F \vee A$

KB3:

KB3: $\neg E \vee D$

KB4 gives:

KB4.1: B

KB4.2: C

KB5:

KB5: F

Negate the Goal

Goal: A

Negated goal:

$\neg A$

Apply Resolution

1. Resolve $\neg A$ with $\neg D \vee \neg F \vee A$
Result: $\neg D \vee \neg F$
2. Resolve $\neg D \vee \neg F$ with $\neg E \vee D$
Result: $\neg F \vee \neg E$
3. Resolve $\neg F \vee \neg E$ with F
Result: $\neg E$
4. Resolve $\neg E$ with $\neg B \vee \neg C \vee E$
Result: $\neg B \vee \neg C$
5. Resolve $\neg B \vee \neg C$ with B
Result: $\neg C$
6. Resolve $\neg C$ with C
Result: $\square$

Since the **empty clause** ($\square$) is obtained, contradiction is found.

Therefore, A is proved.

Horn Clauses and Definite Clauses

These are special types of logical statements used in AI knowledge bases, especially for forward chaining and backward chaining.

1. Horn Clause

A Horn clause is a clause that contains at most one positive literal.

Examples:

$$\neg P \vee \neg Q \vee R$$

Here, only R is positive, so it is a Horn clause.

$$\neg P \vee \neg Q$$

This also is a Horn clause because it has zero positive literals.

But:

$$P \vee Q$$

is not a Horn clause because it has two positive literals.

2. Definite Clause

A definite clause is a Horn clause having exactly one positive literal.

Example:

$$\neg P \vee \neg Q \vee R$$

This can be written as:

$$P \wedge Q \rightarrow R$$

We normally read it as:

If P and Q are true, then R is true.

For example:

$$\text{Rain} \wedge \text{Outside} \rightarrow \text{Wet}$$

Here:

Rain, Outside = conditions/premises

Wet = conclusion

This is a definite clause.

Therefore:

Every definite clause is a Horn clause

but

Every Horn clause is not necessarily a definite clause

Forward Chaining

Forward chaining is a data-driven inference technique in AI that starts with known facts and repeatedly applies rules to derive new facts until the required goal is reached.

Solved Example

Given: Prove Q

Rules and Facts

1. $P \rightarrow Q$
2. $L \wedge M \rightarrow P$
3. $B \wedge L \rightarrow M$

4. $A \wedge P \rightarrow L$

5. $A \wedge B \rightarrow L$

6. $A \text{ — Fact}$

7. $B \text{ — Fact}$

Solution

Start with the known facts:

A, B

Using Rule 5:

$$A \wedge B \rightarrow L$$

Therefore:

L

Using Rule 3:

$$B \wedge L \rightarrow M$$

Therefore:

M

Using Rule 2:

$$L \wedge M \rightarrow P$$

Therefore:

P

Using Rule 1:

$$P \rightarrow Q$$

Therefore:

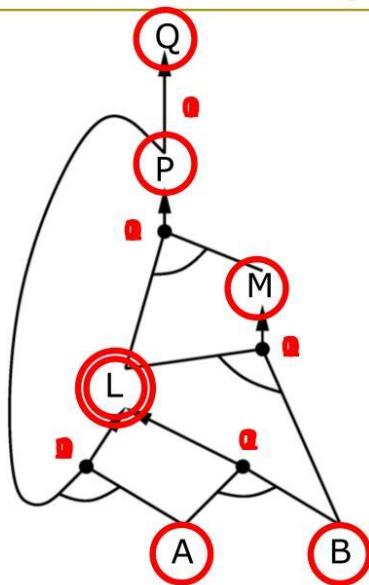
Q

Forward Chaining Sequence

$$\mathbf{A, B \rightarrow L \rightarrow M \rightarrow P \rightarrow Q}$$

Hence, **Q is proved using Forward Chaining.**

Forward chaining example



$P \Rightarrow Q$
 $L \wedge M \Rightarrow P$
 $B \wedge L \Rightarrow M$
 $A \wedge P \Rightarrow L$
 $A \wedge B \Rightarrow L$
 A
 B

Backward Chaining

Backward chaining is a **goal-driven inference technique** in AI that starts with the goal and works backward to find facts or rules that can prove it.

Solved Example

Given:

1. $P \rightarrow Q$
2. $L \wedge M \rightarrow P$
3. $B \wedge L \rightarrow M$
4. $A \wedge P \rightarrow L$
5. $A \wedge B \rightarrow L$
6. A — Fact
7. B — Fact

Goal: Prove Q

Solution

Start with the goal:

Q

To prove Q, from Rule 1 we need:

P

To prove P, from Rule 2 we need:

L and M

To prove M, from Rule 3 we need:

B and L

To prove L, we can use Rule 5, which needs:

A and B

A and B are already **known facts**.

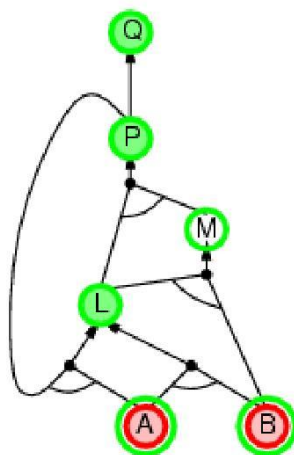
Therefore:

$A, B \Rightarrow L \Rightarrow M \Rightarrow P \Rightarrow Q$

Hence,

Q is proved using Backward Chaining.

Backward chaining example



Key idea

Backward chaining searches:

Goal $\rightarrow$ Required subgoals $\rightarrow$ Known facts

That is why it is called a **goal-driven approach**.

The Wumpus World

The Wumpus world in AI is a 4x4 grid-based problem, which is used to demonstrate the worth of a knowledge-based agent for knowledge representation. It was inspired by the video game Hunt the Wumpus by Gregory Yob in 1973.

Understanding the Wumpus World

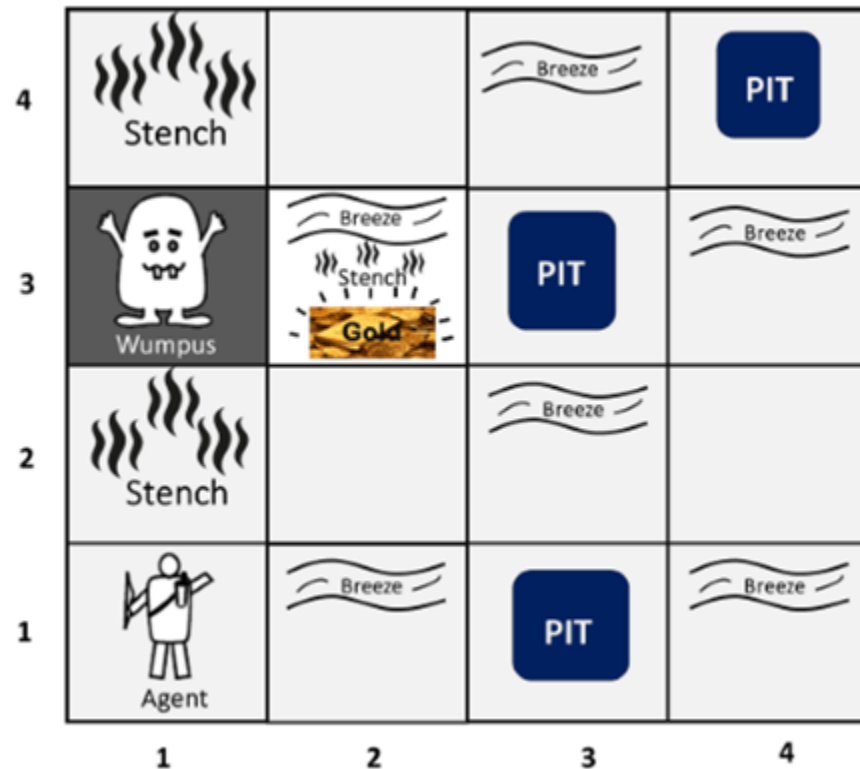
The Wumpus world is a cave that has 4x4 rooms connected with passageways. So there are a total of 16 rooms, which are connected to each other. We have a knowledge-based agent who will go forward in this world. The cave has a room with a beast, which is called Wumpus, that eats anyone who enters the room. The Wumpus can be shot by the agent, but the agent has a single arrow.

In the Wumpus world, there are some Pit rooms which are bottomless, and if the agent falls into a Pit, then he will be stuck there forever. The exciting thing with this cave is that in one room, there is a possibility of finding a heap of gold. So the agent's goal is to find the gold and climb out of the cave without falling into the Pits or being eaten by Wumpus. The agent will get a reward if he comes out with gold, and he will get a penalty if he is eaten by Wumpus or falls in the pit.

Note: Here, Wumpus is static and cannot move.

Following is a sample diagram for representing the Wumpus world. It is showing some rooms with Pits, one room with Wumpus, and one agent at (1,

1) square location of the world.



Sensory Information in Wumpus World

The agent receives percepts from the environment that help it navigate:

- **Stench:** Rooms adjacent (not diagonal) to the Wumpus contain a stench.
- **Breeze:** Rooms adjacent to pits are breezy.
- **Glitter:** A room containing gold glitter.
- **Scream:** If the Wumpus is killed, it emits a scream heard throughout the cave.
- **Bump:** The agent perceives a bump when it walks into a wall.
- The Wumpus can be killed by the agent if the agent is facing it, and the Wumpus will emit a horrible scream, which can be heard anywhere in the cave.

PEAS Description of the Wumpus world

PEAS means Performance, Environment, Actuators, and Sensors. It explains the Wumpus world. We have given a PEAS description as follows:

Performance Measure

- +1000 reward points if the agent comes out of the cave with the gold.

- -1000 points penalty for being eaten by the Wumpus or falling into the pit.
- -1 for each action, and -10 for using an arrow.
- The game ends if either agent dies or comes out of the cave.

Environment

- A 4x4 grid of rooms.
- The agent initially is in room square [1, 1], facing toward the right.
- The location of Wumpus and gold is chosen randomly except for the first square [1,1].
- Each square of the cave can be a pit with probability 0.2, except the first square.

Actuators:

- Left turn
- Right turn
- Move forward
- Grab
- Release
- Shoot

Sensors

- The agent will perceive the stench if he is in the room adjacent to the Wumpus. (Not diagonally).
- The agent will perceive the breeze if he is in the room directly adjacent to the Pit.
- The agent will perceive the glitter in the room where the gold is present.
- The agent will perceive the bump if he walks into a wall.
- When the Wumpus is shot, it emits a horrible scream which can be perceived anywhere in the cave.
- These percepts can be represented as a five-element list, in which we will have different indicators for each sensor.

- For example, if the agent perceives stench, breeze, but no glitter, no bump, and no scream, then it can be represented as [Stench, Breeze, None, None, None].

Properties of the Wumpus World

Let's take a look at the properties of the Wumpus World:

- **Partially Observable:** The Wumpus world is partially observable because the agent can only perceive the close environment, such as an adjacent room.
- **Deterministic:** It is deterministic, as the result and outcome of the world are already known.
- **Sequential:** The order is important, so it is sequential.
- **Static:** It is static as the Wumpus and Pits are not moving.
- **Discrete:** The environment is discrete.
- **Single Agent:** The environment is a single agent, as we have one agent only, and Wumpus is not considered an agent.

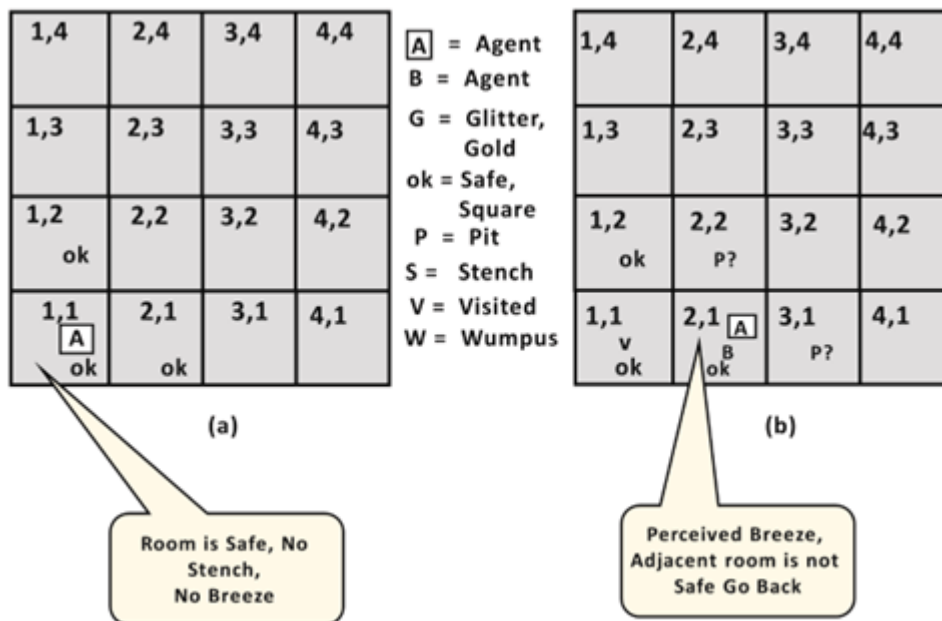
Exploring the Wumpus world

Now we will explore the Wumpus world and determine how the agent will find its goal by applying logical reasoning.

Step 1: Starting Position

Initially, the agent is in the first room or on the square [1,1], and we already know that this room is safe for the agent, so to represent in the diagram below (a) that room is safe, we will add the symbol OK. Symbol A is used to represent the agent, symbol B for the breeze, G for Glitter or gold, V for the visited room, P for pits, and W for Wumpus.

At Room [1,1], the agent does not feel any breeze or any Stench, which means the adjacent squares are also OK.



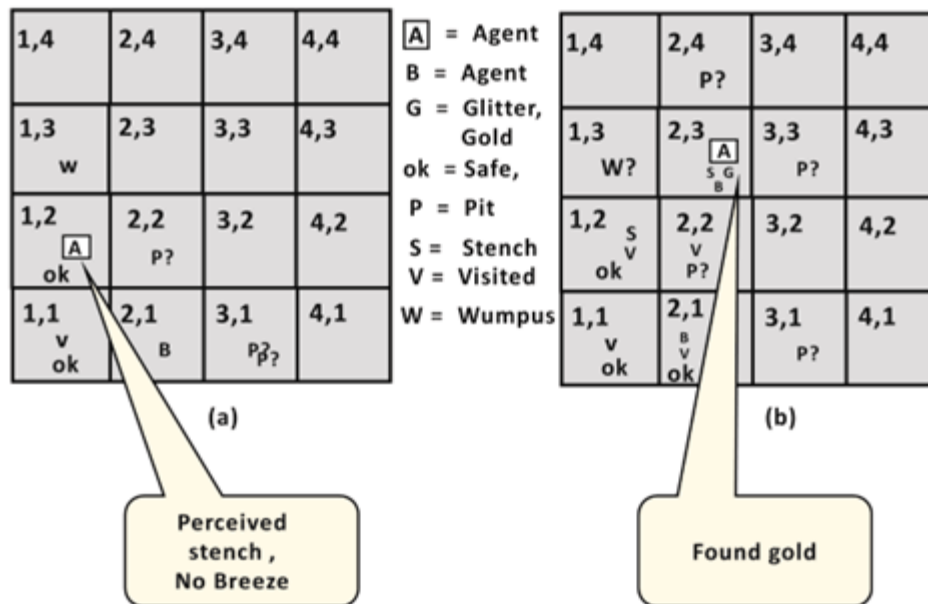
Step 2: Moving to [2,1]

Now the agent needs to move forward, so it will either move to [1, 2] or [2,1]. Let's suppose the agent moves to the room [2, 1]. In this room, the agent perceives some breeze, which means Pit is around this room. The pit can be in [3, 1], or [2,2], so we will add the symbol P? to say that, is this Pit room?

Now the agent will stop and think and will not make any harmful move. The agent will go back to the [1, 1] room. The rooms [1,1] and [2,1] are visited by the agent, so we will use the symbol V to represent the visited squares.

Step 3: Moving to [1,2]

At the third step, the agent will move to the room [1,2], which is OK. In the room [1,2], the agent perceives a stench, which means there must be a Wumpus nearby. But Wumpus cannot be in the room [1,1] as by the rules of the game, and also not in [2,2] (Agent had not detected any stench when he was at [2,1]). Therefore, the agent infers that Wumpus is in the room [1,3], and in the current state, there is no breeze, which means in [2,2] there is no Pit and no Wumpus. So it is safe, and we will mark it OK, and the agent moves further in [2,2].



Step 4: Moving to [2,3]

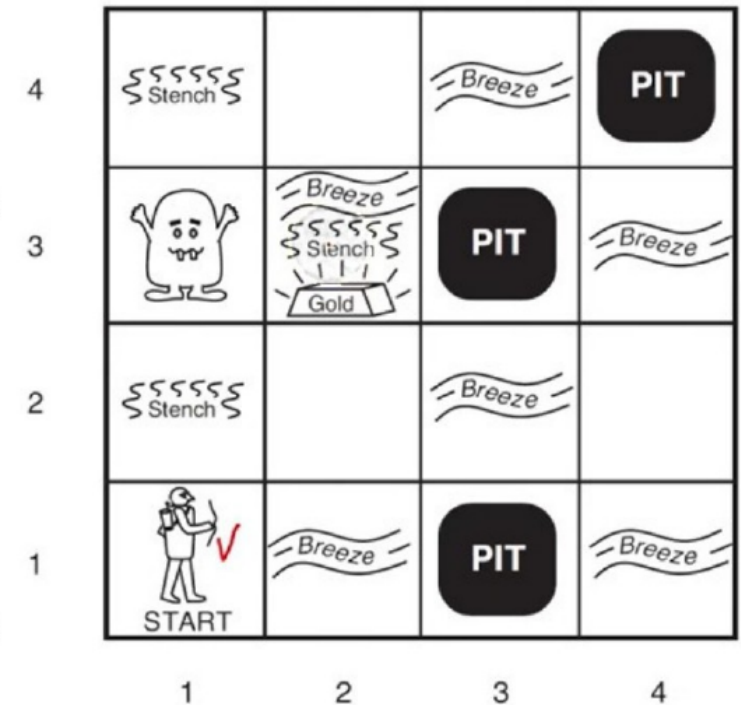
At room [2,2], there is no stench and no breezes present, so let's suppose the agent decides to move to [2,3]. At room [2,3], the agent perceives glitter, so it should grab the gold and climb out of the cave.

Conclusion

We have learnt a lot about the Wumpus world. It was inspired by the video game **Hunt the Wumpus** by Gregory Yob in 1973. It is a simple world example to illustrate the worth of a knowledge-based agent and to represent knowledge representation. We studied the PEAS rule in the Wumpus World, which means Performance measure, Environment, Actuators, and Sensors. We also studied the four steps of the agent in the exploration of the Wumpus World.

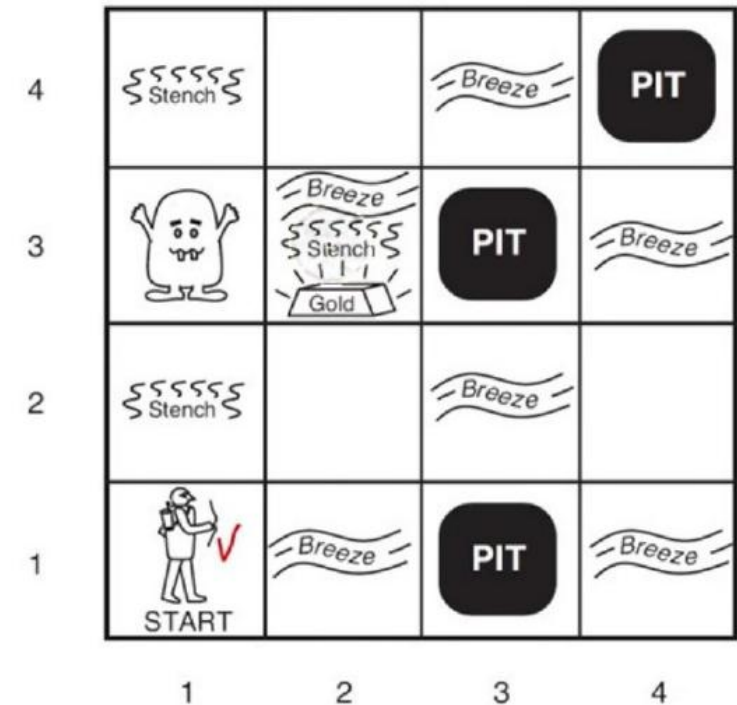
The Wumpus World

- Wumpus World is a classic problem in Artificial Intelligence used to explain how a knowledge-based agent represents knowledge and reasons using logic.
- The world is usually represented as a 4×4 grid containing:
 - Wumpus** — a dangerous creature
 - Pits** — dangerous holes
 - Gold** — the goal
 - Agent** — moves through the environment
- The main objective of the agent is to find the gold and return safely without falling into a pit or being eaten by the Wumpus.



Wumpus World Example in AI

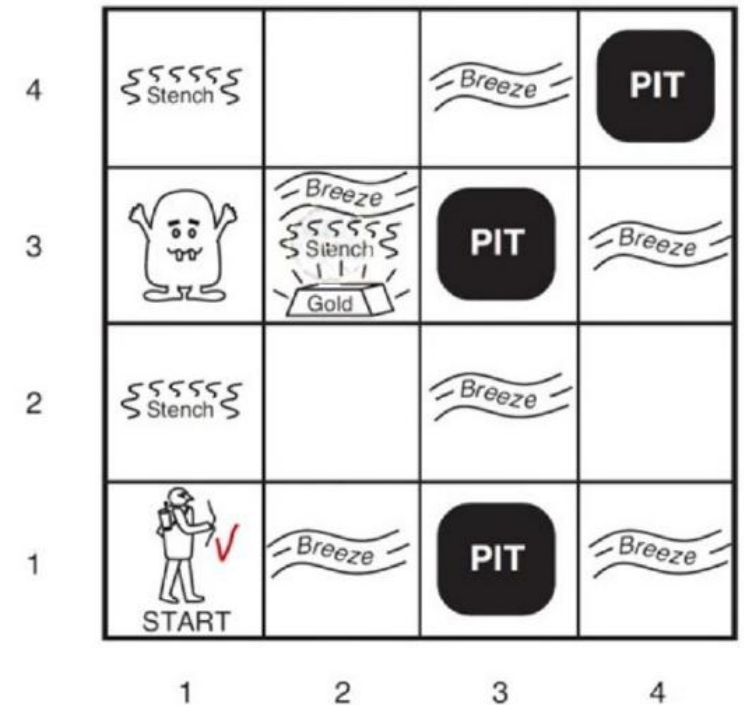
- The Wumpus can be shot by the agent, but the agent has a single arrow.
- Some rooms contain bottomless pits that trap any agent that wanders into the room.
- Occasionally, there is a heap of gold in a room.
- The goal is to collect the gold and exit the world without being eaten.



PEAS description of Wumpus world:

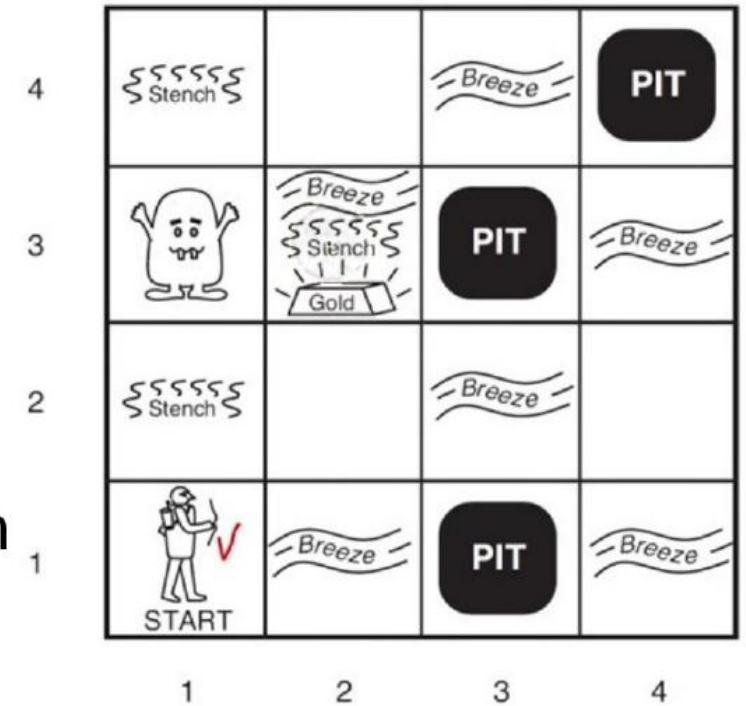
Performance measure

- +1000 reward points if the agent comes out of the cave with the gold.
- -1000 points penalty for being eaten by the Wumpus or falling into the pit.
- -1 for each action, and -10 for using an arrow.
- The game ends if either agent dies or came out of the cave.



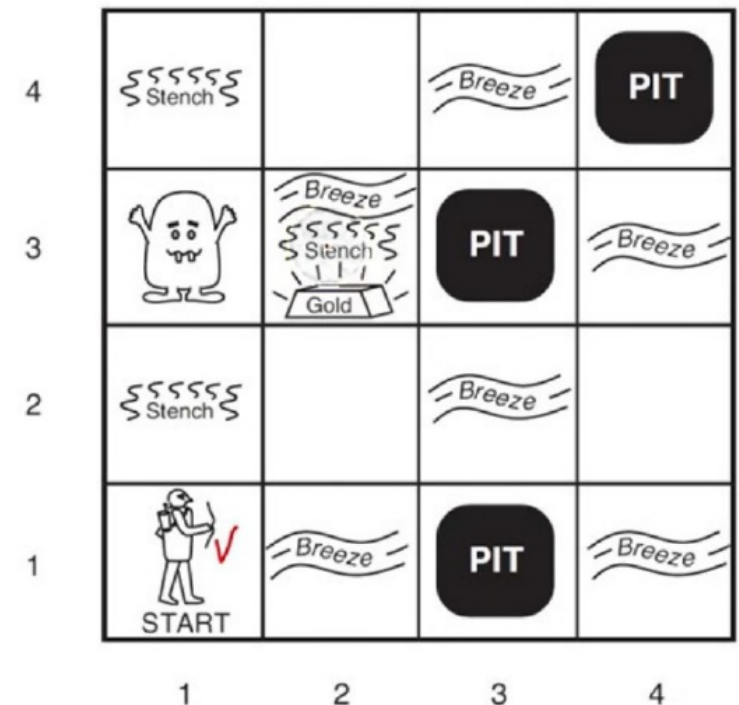
Environment:

- A 4*4 grid of rooms.
- The agent initially in room square [1, 1], facing toward the right.
- Location of Wumpus and gold are chosen randomly except the first square [1, 1].



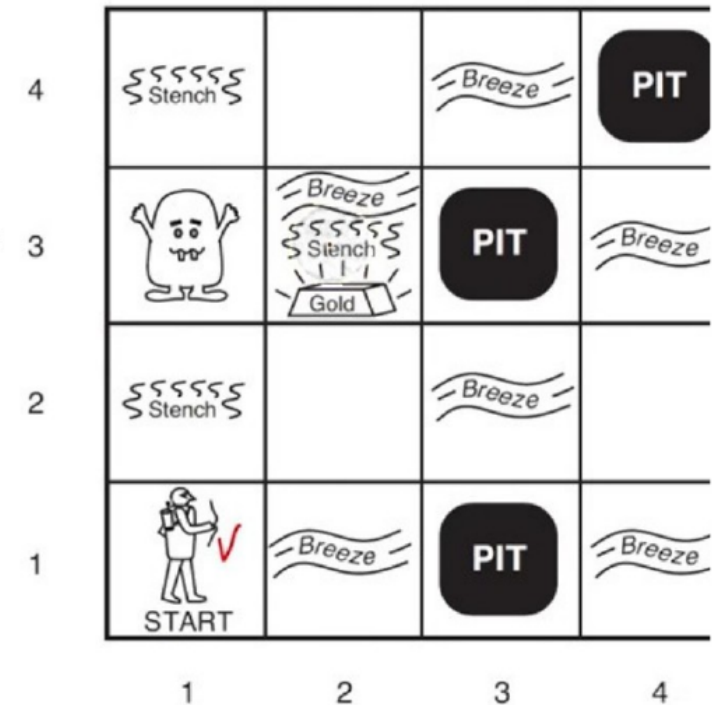
Actions/Actuators:

- The agent can move Forward, TurnLeft by 90°, or TurnRight by 90°.
- The action **Grab** can be used to pick up the gold if it is in the same square as the agent.
- The action **Shoot** can be used to fire an arrow in a straight line in the direction the agent is facing.
- The action **Climb** can be used to climb out of the cave, but only from square [1,1].



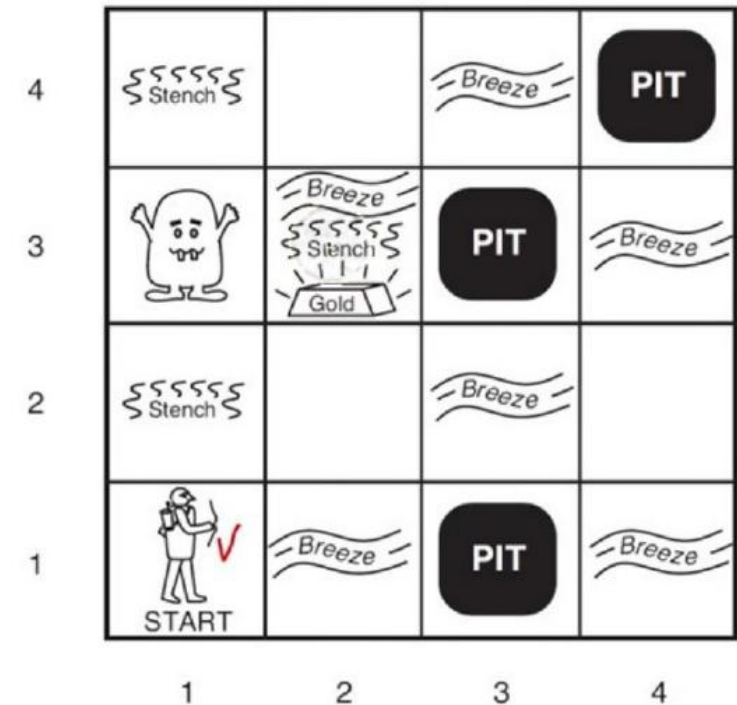
Sensors: The agent has five sensors:

- In the square containing the Wumpus and in the directly (not diagonally) adjacent squares, the agent will perceive a **Stench**.
- In the squares directly adjacent to a pit, the agent will perceive a **Breeze**.
- In the square where the gold is, the agent will perceive a **Glitter**.
- When an agent walks into a wall, it will perceive a **Bump**.
- When the Wumpus is killed, it emits a woeful **Scream** that can be perceived anywhere in the cave.



Percepts in Wumpus World

- The percepts will be given to the agent program in the form of a list of five symbols.
- For example:
- If there is a stench and a breeze, but no glitter, bump, or scream,
- The agent program will get
- [Stench, Breeze, None, None, None].



1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2	2,2	3,2	4,2
OK			
1,1 A OK	2,1 OK	3,1	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

After Second move, with percept [None, None, None, None, None].

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 A V OK	2,1 V OK	3,1 P?	4,1

- After the third move, with percept [Stench, None, None, None, None]
- After the Fourth move, with percept [None, None, None, None, None]

1,4	2,4	3,4	4,4
1,3 W!	2,3	3,3	4,3
1,2 A S OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4	3,4	4,4
1,3 W!	2,3 OK	3,3	4,3
1,2 S OK	2,2 A OK	3,2 OK	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

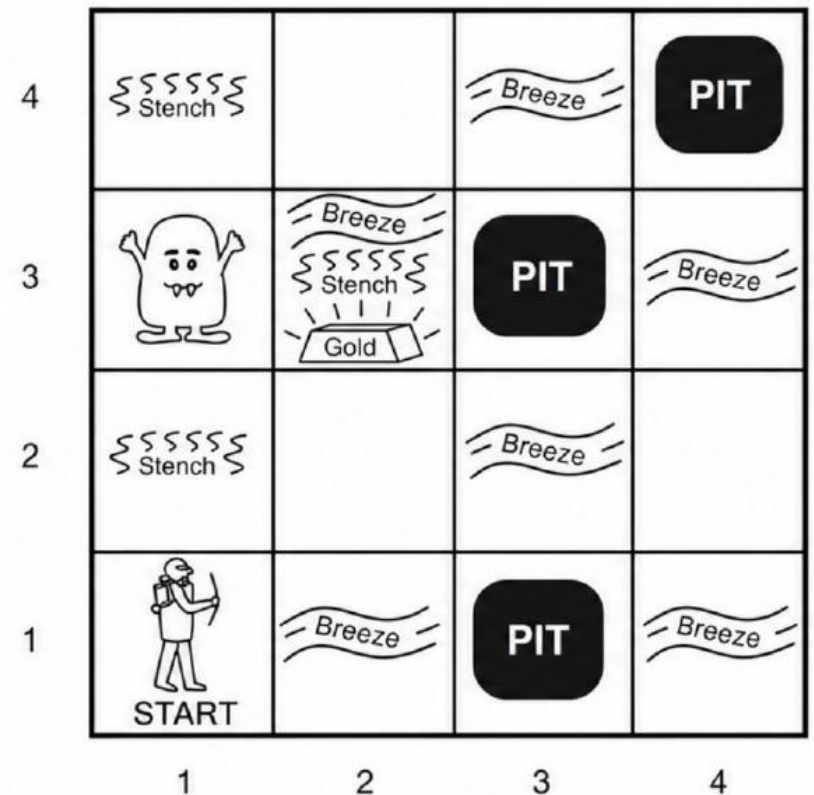
- After the fifth move, with percept [Stench, Breeze, Glitter, None, None]

1,4	2,4	3,4	4,4
1,3 W!	2,3 OK	3,3	4,3
1,2 S OK	2,2 A OK	3,2 OK	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

A = Agent
B = Breeze
G = Glitter, Gold
OK = Safe square
P = Pit
S = Stench
V = Visited
W = Wumpus

1,4	2,4 P?	3,4	4,4
1,3 W!	2,3 A S G B	3,3 P?	4,3
1,2 S V OK	2,2 V OK	3,2 OK	4,2
1,1 V OK	2,1 B V OK	3,1 P!	4,1

- Knowledge bases consist of **sentences**.
- Sentences are expressed according to the **syntax of logic**.
- **Example:** “ $x + y = 4$ ” is a well-formed sentence, whereas “ $x4y+ =$ ” is not
- The **semantics** defines the **truth** of each sentence with respect to each **possible world (model)**.



- **Logical entailment** between sentences is that a **sentence follows logically from another sentence**.
- Mathematical notation: $\alpha \models \beta$ (sentence α **entails** the sentence β .)
- The formal definition of entailment is this: $\alpha \models \beta$ if and only if, in every model in which α is true, β is also true.

$$\alpha \models \beta \text{ if and only if } M(\alpha) \subseteq M(\beta)$$

- $\alpha : x + y = 4$
- $\beta : x * y \geq 2$
- For all $x, y \leq 3$

(1, 1), (1, 2), (1, 3), (2, 1), (2, 2), (2, 3), (3, 1), (3, 2) and (3, 3)

α is true when x is 1 and y is 3, x is 2 and y is 2 and x is 3 and y is 1.

A model is simply a set of values that makes a statement true.

We can apply the same kind of analysis to the wumpus-world reasoning example.

Consider the situation in Figure:
the agent has detected nothing in [1,1] and a breeze in [2,1].

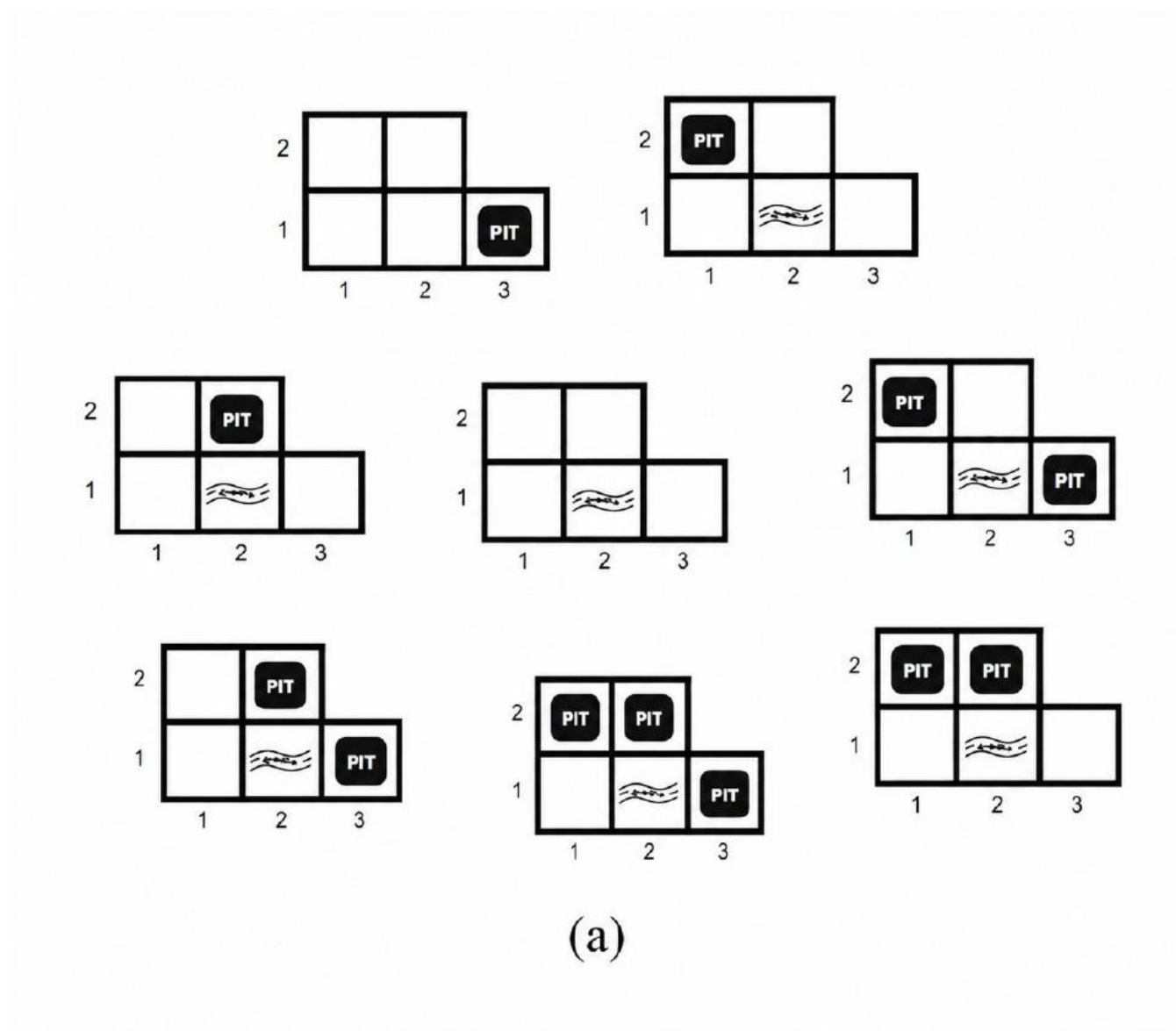
1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2	3,2	4,2
1,1 V OK	2,1 A B OK	3,1	4,1

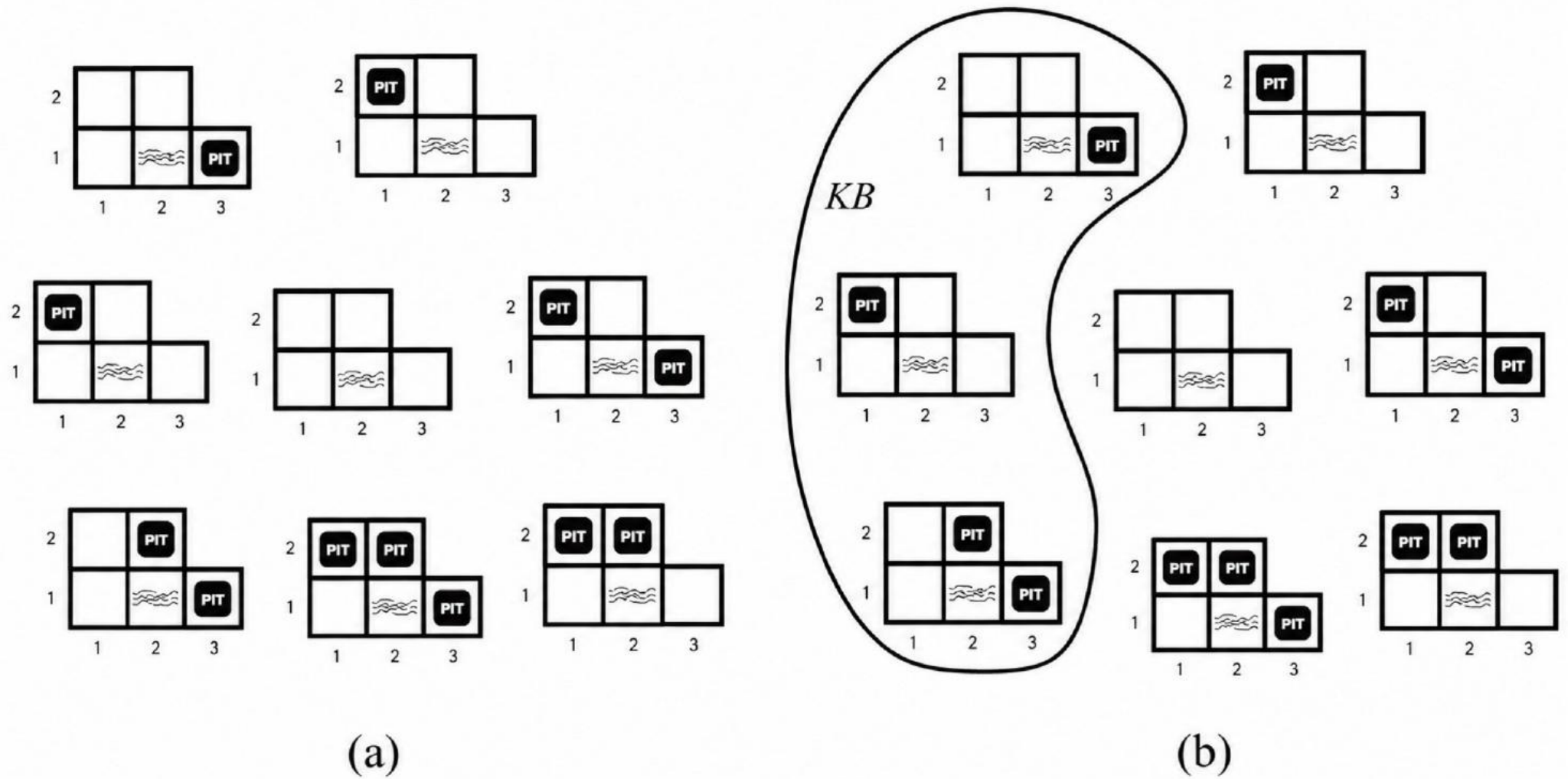
- These percepts, combined with the agent's knowledge of the rules of the wumpus world, constitute the **KB**.
- The agent is interested in whether the adjacent squares [1,2], [2,2], and [3,1] contain pits.
- Each of the three squares might or might not contain a pit.

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 A B OK	3,1	4,1

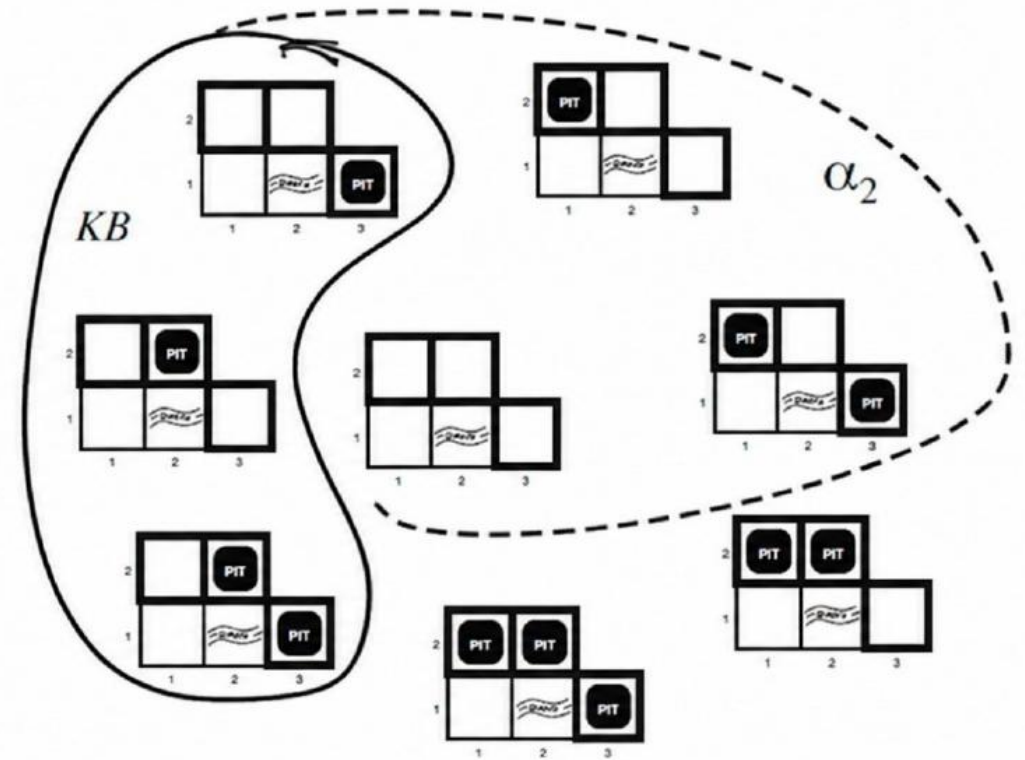
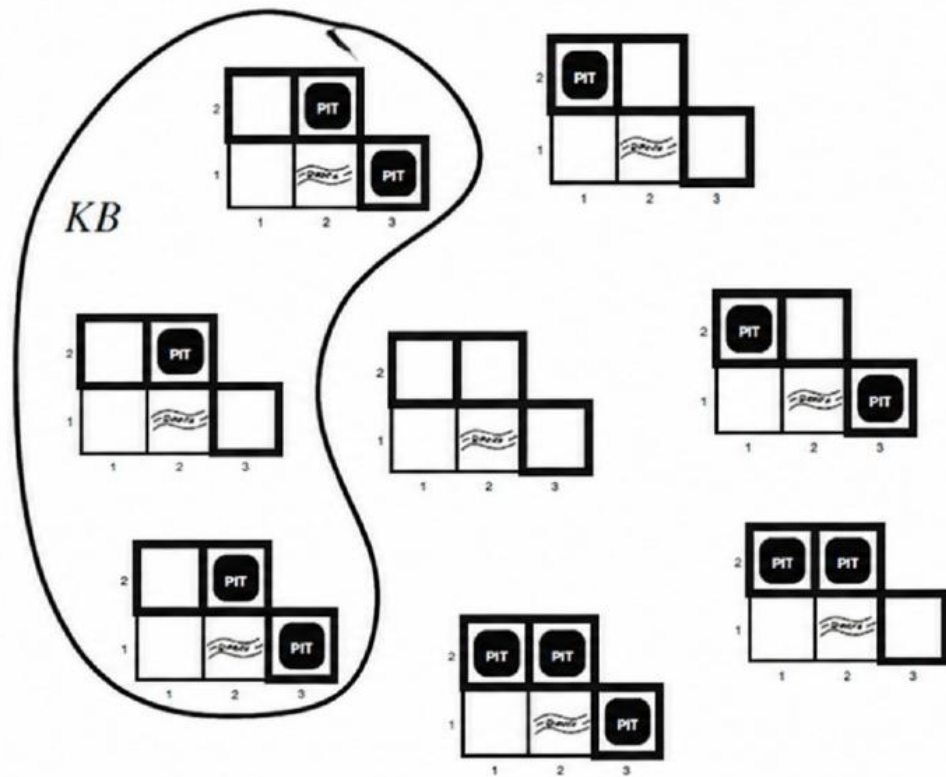
- These percepts, combined with the agent's knowledge of the rules of the wumpus world, constitute the **KB**.
- The agent is interested in whether the adjacent squares [1,2], [2,2], and [3,1] contain pits.
- Each of the three squares might or might not contain a pit.

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 OK	3,2	4,2
1,1 V OK	2,1 A B OK	3,1	4,1





- Let us consider two possible conclusions:
 - α_1 = “There is no pit in [1,2].”
 - α_2 = “There is no pit in [2,2].”
 - **$\neg \text{KB} \models \alpha_1$**
 - **$\neg \text{KB} \models \alpha_2$**



A simple knowledge base

- To construct a knowledge base for the wumpus world.
- We need the following symbols for each $[x, y]$ location:
 - $P_{x,y}$ is true if there is a pit in $[x, y]$.
 - $W_{x,y}$ is true if there is a wumpus in $[x, y]$, dead or alive.
 - $B_{x,y}$ is true if the agent perceives a breeze in $[x, y]$.
 - $S_{x,y}$ is true if the agent perceives a stench in $[x, y]$.

The model-checking algorithm does this:

- 1. Generate all possible models.**
- 2. Find the models where KB is true.**
- 3. Check whether α is true in all those models.**
- 4. If yes:**
- 5. $KB \models \alpha$**

Then the agent can conclude α .

Wumpus World Model Checking / Truth Table Approach

- The sentences are labeled as R_i :
- There is no pit in [1,1]:
 - **R1** : $\neg P_{1,1}$
- A square is breezy if there is a pit in a neighboring square:
 - **R2** : $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
 - **R3** : $B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
- The preceding sentences are true in all wumpus worlds.
- Breeze percepts for the first two squares visited
 - **R4** : $\neg B_{1,1}$
 - **R5** : $B_{2,1}$

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 B?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

- R1 : $\neg P_{1,1}$
- R2 : $B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
- R3 : $B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
- R4 : $\neg B_{1,1}$
- R5 : $B_{2,1}$

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 P?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1

$B_{1,1}$	$B_{2,1}$	$P_{1,1}$	$P_{1,2}$	$P_{2,1}$	$P_{2,2}$	$P_{3,1}$	KB
false	false	false	false	false	false	false	false
false	false	false	false	false	false	true	false
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
false	true	false	false	false	false	false	false
false	true	false	false	false	false	true	<u>true</u>
false	true	false	false	false	true	false	<u>true</u>
false	true	false	false	false	true	true	<u>true</u>
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
false	true	false	false	false	true	true	<u>true</u>
false	true	false	false	true	false	false	false
false	true	false	false	$\vdots$	$\vdots$	$\vdots$	$\vdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
true	true	true	true	true	true	true	false

Wumpus World Model Checking / Truth Table Approach

- The sentences are labeled as R_i :
- There is no pit in [1,1]:
 - $R1 : \neg P_{1,1}$
- A square is breezy if there is a pit in a neighboring square:
 - $R2 : B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$
 - $R3 : B_{2,1} \Leftrightarrow (P_{1,1} \vee P_{2,2} \vee P_{3,1})$
- The preceding sentences are true in all wumpus worlds.
- Breeze percepts for the first two squares visited
 - $R4 : \neg B_{1,1}$
 - $R5 : B_{2,1}$

1,4	2,4	3,4	4,4
1,3	2,3	3,3	4,3
1,2 OK	2,2 B?	3,2	4,2
1,1 V OK	2,1 A B OK	3,1 P?	4,1